

# 项目相关及配置

## 项目地址

[blapapple/XIUOS Proj\\_IntelligentStudyRoom](#): 专业实习实验XIUOS仓库，用于小组合作。

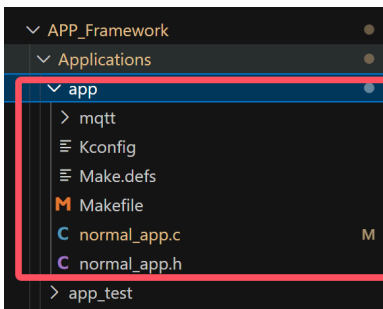
## 参考资料

[下载](#) | [XIUOS 矽璞工业物联操作系统](#)

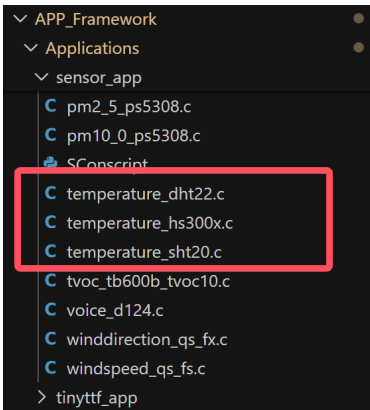
## 文件结构

APP\_Framework目录下的applications是应用代码，包含main.c和其他demo代码，可以参考demo进行开发，Framework目录下是各种驱动代码，在之后可以参考其他驱动进行驱动开发。

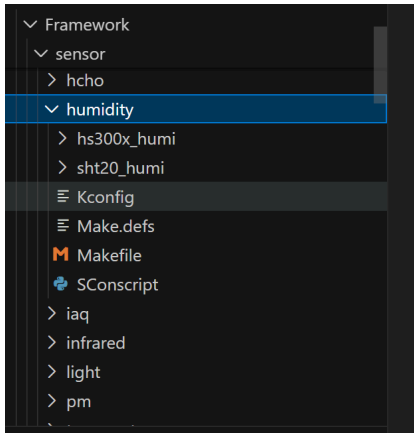
本项目中，在applications目录下添加了app目录，存放项目应用层代码。



在sensorapp下添加了用于测试的几个传感器应用层程序。



在Framework下的sensor驱动层添加了温湿度传感器和光照传感器的驱动。



# 环境配置

## LINUX-wsl2配置环境

在wsl2环境下，参考《开发手册》p33进行环境配置

```
sudo apt install build-essential pkg-config
sudo apt install gcc make libncurses5-dev openssl libssl-dev bison flex libelf-
dev autoconf libtool gperf libc6-dev git nodejs npm

#以下是安装kconfig，找个自己喜欢的文件夹安装就行了
mkdir kfrontends && cd kfrontends
git clone https://gitlink.org.cn/xuos/kconfig-frontends.git
./xs_build.sh
```

在安装交叉编译链时，参考github仓库推荐用法，省去了找版本和解压的操作，使用xpm安装了对应版本的编译链。安装xpm时先安装最新的node和npm。

```
$ xpm install --global @gnu-mcu-eclipse/riscv-none-gcc@8.2.0-2.1.1
```

在自己喜欢的地方clone源码

```
git clone https://gitlink.org.cn/xuos/xiuos.git
```

如果是使用xpm下载的交叉编译链，在这个目录的这个文件把对应路径修改掉

```
Ubiquitous/XiZi_IIoT/board/edu-riscv64/config.mk
```

如我的路径修改为下面这个路径，这个路径可以在下载这个库时观察到，若找不到可以去问ai如何看xpm的下载路径

```
export CROSS_COMPILE ?=/home/blapapple/.local/xPacks/@gnu-mcu-eclipse/riscv-none-
gcc/8.2.0-2.1.1/.content/bin/riscv-none-embed-
```

后续直接参考《手册》

## WINDOWS

使用python3.11，安装paho.mqtt库

```
pip install paho.mqtt
```

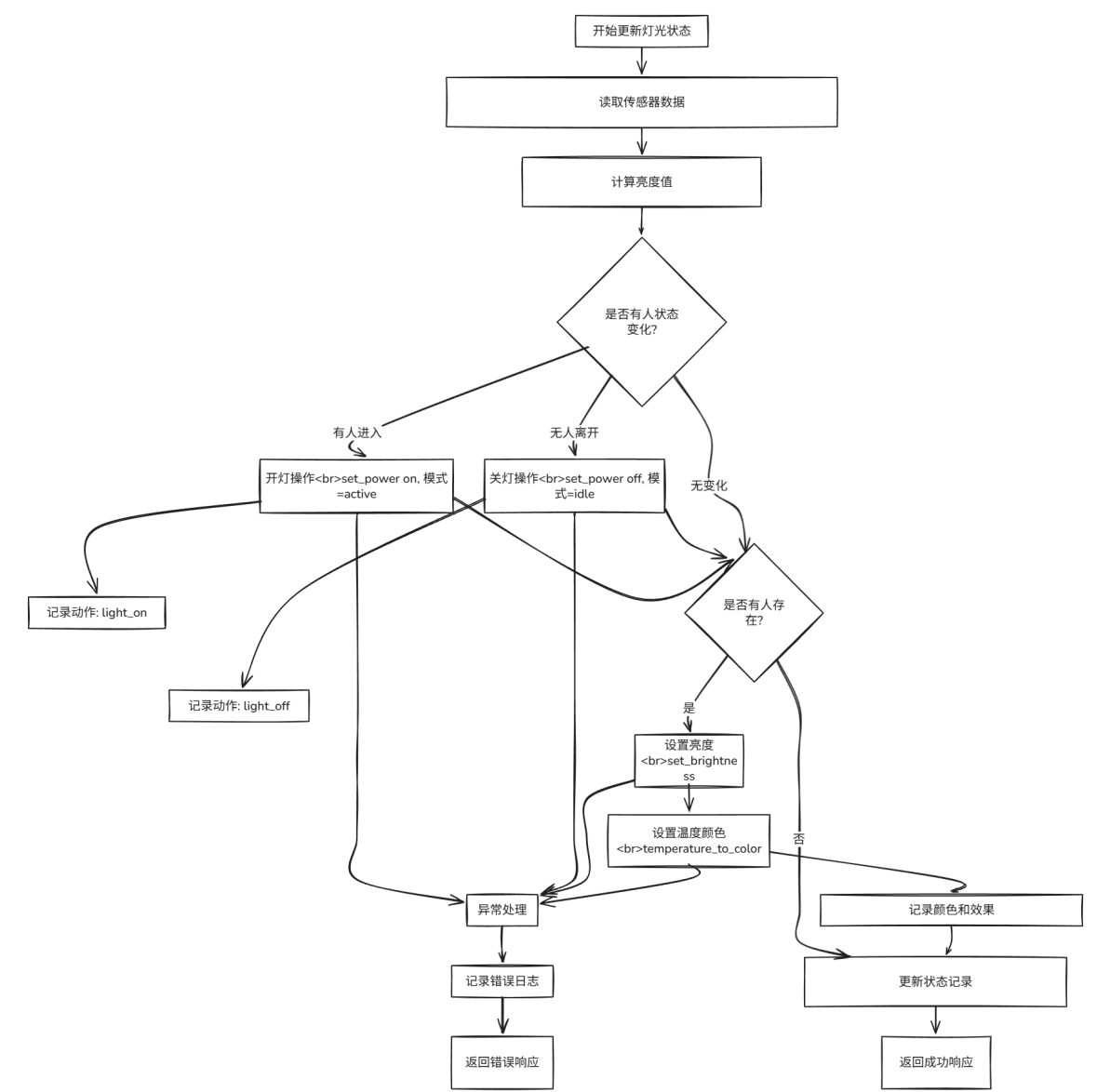
安装mosquitto，进入mosquitto下载网址：<https://mosquitto.org/download/>

安装window版本即可

# 系统实现

## 精准控制——服务端开发

系统的服务端由两部分组成，一部分是MQTT的客户端，一部分是控制智能灯泡的逻辑，控制灯泡的协议是厂家给出，按照协议通过WIFI向灯泡发命令即可控制灯泡亮度和颜色。



以下是控制灯泡的代码部分。

```
def update_light_state(self, sensor_data: Dict) -> Dict:
    """根据传感器数据更新灯泡状态"""
    person_present = sensor_data.get('person_present', False)
    light_intensity = sensor_data.get('light_intensity', 0)
    temperature = sensor_data.get('temperature', 22.0)
    humidity = sensor_data.get('humidity', 50.0)

    brightness = self.calculate_brightness(light_intensity, person_present)
    response = {}

    try:
        # 控制开关
        if person_present and not self.last_person_state:
            # 有人进入：开灯
```

```

self.light_client.set_power("on", "smooth", 300)
self.current_mode = "active"
response['action'] = "light_on"
logger.info("检测到有人，开灯")

elif not person_present and self.last_person_state:
    # 无人：关灯
    self.light_client.set_power("off", "smooth", 1000)
    self.current_mode = "idle"
    response['action'] = "light_off"
    logger.info("无人，关灯")

# 如果有人，设置亮度和颜色效果
if person_present:
    # 设置亮度
    self.light_client.set_brightness(brightness, "smooth", 500)
    response['brightness'] = brightness

    # 停止之前的流动效果
    # self.light_client.stop_color_flow()

    # 根据温湿度创建动态效果
    if humidity > self.HUMIDITY_THRESHOLDS["humid"] or temperature >
self.TEMP_THRESHOLDS["warm"]:
        # 高湿度或高温度：启动流动效果
        # flow_expression = self.create_weather_flow(temperature,
humidity)

        # self.light_client.start_color_flow(flow_expression)
        self.current_mode = "weather_flow"
        response['effect'] = "weather_flow"
        logger.info(f"温度: {temperature}°C, 湿度: {humidity}%")
    else:
        # 正常情况：固定颜色
        color = self.temperature_to_color(temperature)
        self.light_client.set_rgb(color[0], color[1], color[2],
"smooth", 800)

        self.current_mode = "temperature_color"
        response['color'] = color
        response['effect'] = "temperature_based"
        logger.info(f"设置温度颜色: RGB{color}, 温度: {temperature}°C")

self.last_person_state = person_present
self.last_update_time = time.time()
response['status'] = 'success'
response['mode'] = self.current_mode

except Exception as e:
    logger.error(f"控制灯泡失败: {str(e)}")
    response['status'] = 'error'
    response['message'] = str(e)

return response

```

以上代码在服务端接收到正确格式的MQTT消息后调用，接收消息的代码如下所示。



```

def _on_message(self, client: mqtt.Client, userdata: Any, msg: mqtt.MQTTMessage):
    logger.info(f"Received message from topic {msg.topic}")

    try:
        payload = msg.payload.decode('utf-8')
        message_data = json.loads(payload)

        # 处理设备状态消息
        if 'sensor_data' in message_data:
            sensor_data = message_data['sensor_data']
            logger.info(f"处理传感器数据：有人={
{sensor_data.get('person_present')}}, "
                        f"光照={sensor_data.get('light_intensity')}lx, "
                        f"温度={sensor_data.get('temperature')}°C, "
                        f"湿度={sensor_data.get('humidity')}%")

            # 使用灯泡状态管理器处理传感器数据
            if self.light_state_manager:
                result =
self.light_state_manager.update_light_state(sensor_data)
                logger.info(f"灯泡控制结果：{result}")

    except Exception as e:
        logger.error(f"处理MQTT消息失败：{str(e)}")

```

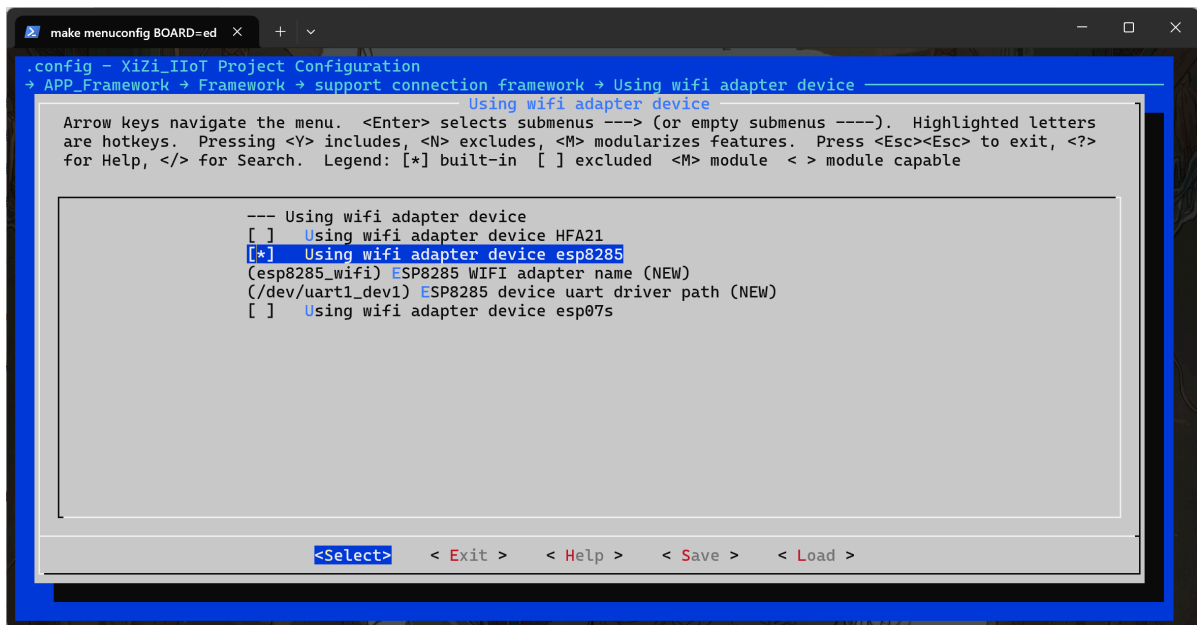
## 泛在互联——通讯开发

XiUOS的“泛在互联”网络框架采用以网络适配器为中心的抽象方式，发送与接收数据时，只需要像Linuxsocket网络通信一样，仅关注发送与接收的数据，而不用把重点放在所使用的具体协议上。这种抽象方式有效屏蔽了种类繁多的网络协议细节，对外提供统一的网络接口，简化了基于嵌入式操作系统的网络开发。网络框架对各类协议的网络设备进行了抽象，即一个网络协议设备，被抽象为一个Adapter。

Adapter可以被设计为采用类似面向对象的方法，针对不同的网络协议，扩展其数据成员与接口，从而为种类繁多的网络协议实现统一的管理架构。在网络应用开发的过程中，只需要使用对应网络协议的Adapter实例，而无需关心网络协议的硬件细节，从而实现了网络数据收发功能与底层硬件的解耦。

首先参考《手册》p502页进行menuconfig的wifi配置

注意，下图此处也需要配置，在《手册》中未指明



在XIUOS/APP\_Framework/Framework/connection/wifi/esp8285\_wifi/esp8285\_wifi.c 处找到如下函数。

```
static int Esp8285WifiSetup(struct Adapter *adapter)
```

分析可知，在传入的Adapter里的成员变量“void \*adapter\_param”写入wifi的ssid和密码即可连接上WiFi。

在main.c中添加如下函数，编译后测试

```
int WifiInitAndConnect(void)
{
    int ret = 0;

    // 1. 查找Wi-Fi适配器
    struct Adapter* adapter = AdapterDeviceFindByName(ADAPTER_WIFI_NAME);
    if (!adapter) {
        printf("Wi-Fi adapter not found!\n");
        return -1;
    }

    // 2. 打开Wi-Fi设备
    ret = AdapterDeviceOpen(adapter);
    if (ret != 0) {
        printf("Failed to open Wi-Fi device! Error: %d\n", ret);
        return ret;
    }
    printf("Wi-Fi device opened successfully.\n");

    // 3. 配置Wi-Fi连接参数（使用宏定义）
    static struct WifiParam param;
    memset(&param, 0, sizeof(struct WifiParam));
    strncpy((char *)param.wifi_ssid, WIFI_SSID, sizeof(param.wifi_ssid) - 1);
    strncpy((char *)param.wifi_pwd, WIFI_PASSWORD, sizeof(param.wifi_pwd) - 1);

    // 确保字符串以null结尾
    param.wifi_ssid[sizeof(param.wifi_ssid) - 1] = '\0';
    param.wifi_pwd[sizeof(param.wifi_pwd) - 1] = '\0';
}
```

```

    adapter->adapter_param = &param;

    // 4. 执行连接操作
    printf("Connecting to Wi-Fi: %s...\n", WIFI_SSID);
    ret = AdapterDeviceSetUp(adapter);
    if (ret != 0) {
        printf("Wi-Fi connection failed! Error: %d\n", ret);
        // 关闭设备以防资源泄漏
        AdapterDeviceClose(adapter);
        return ret;
    }

    printf("Wi-Fi connected successfully to: %s\n", WIFI_SSID);

    // 5. 可选：等待一段时间确保连接稳定
    PrivTaskDelay(3000);

    return ret;
}

#ifdef OTA_BY_PLATFORM
extern int OtaTask(void);
#endif

#ifdef APPLICATION_WEBSERVER
extern int webserver(void);
#endif

int main(void)
{
    printf("\nHello, world!\n");
    FrameworkInit();
#ifdef APPLICATION_OTA
    ApplicationOtaTaskInit();
#endif

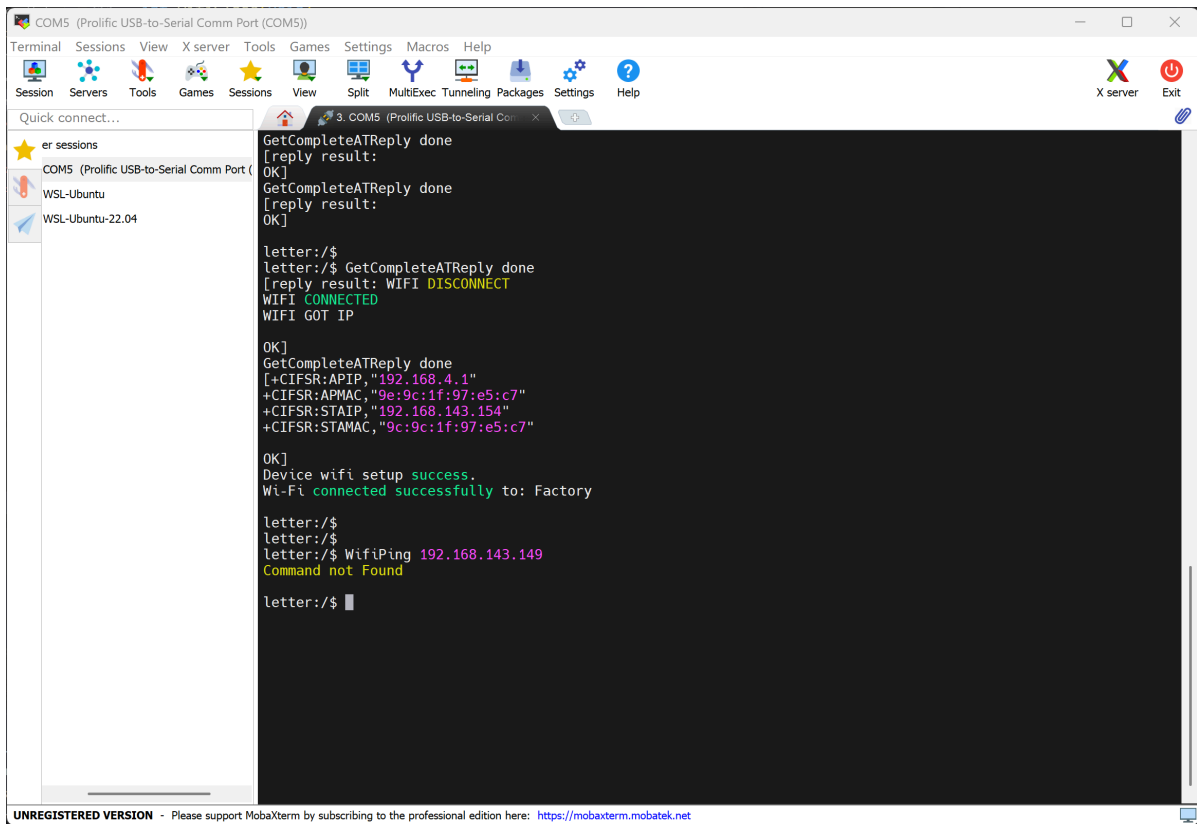
#ifdef OTA_BY_PLATFORM
    OtaTask();
#endif

#ifdef APPLICATION_WEBSERVER
    webserver();
#endif

    wifiInitAndConnect();
    return 0;
}

```

刷写后重启板子，结果如下所示，成功连接WiFi



```
COM5 (Prolific USB-to-Serial Comm Port (COM5))
Terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help

Quick connect...
er sessions
COM5 (Prolific USB-to-Serial Comm Port)
WSL-Ubuntu
WSL-Ubuntu-22.04

GetCompleteATReply done
[reply result: OK]
GetCompleteATReply done
[reply result: OK]

letter:/$
letter:/$ GetCompleteATReply done
[reply result: WIFI DISCONNECT]
WIFI CONNECTED
WIFI GOT IP

OK]
GetCompleteATReply done
[+CIFSR:APIP,"192.168.4.1"
+CIFSR:APMAC,"9e:9c:1f:97:e5:c7"
+CIFSR:STAIP,"192.168.143.154"
+CIFSR:STAMAC,"9c:9c:1f:97:e5:c7"

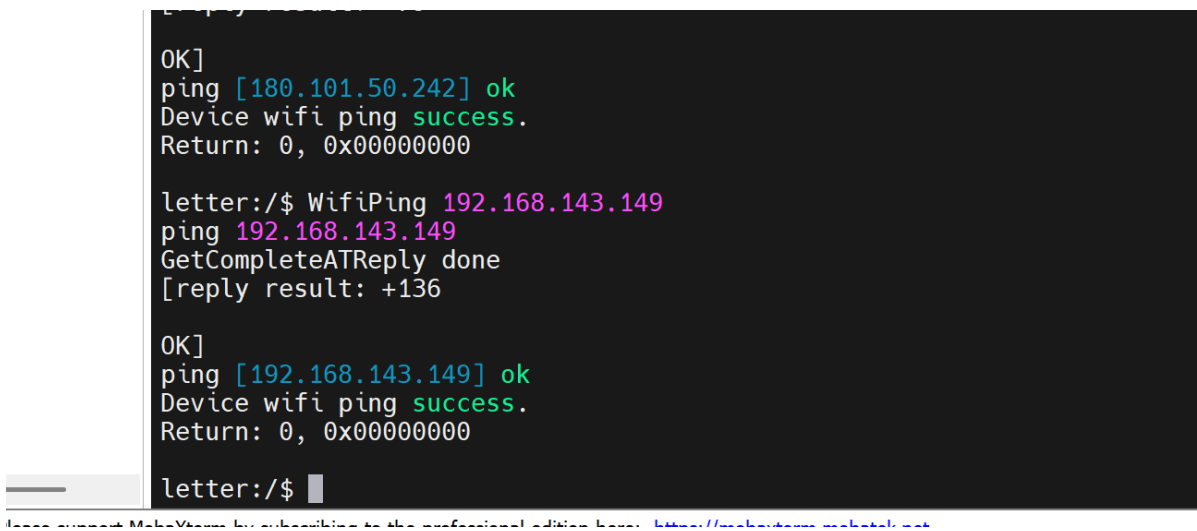
OK]
Device wifi setup success.
Wi-Fi connected successfully to: Factory

letter:/$
letter:/$
letter:/$ WifiPing 192.168.143.149
Command not Found

letter:/$
```

UNREGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.net>

连接上热点的情况下，ping 公网 180.101.50.242 和局域网pc端 192.168.143.149 成功



```
OK]
ping [180.101.50.242] ok
Device wifi ping success.
Return: 0, 0x00000000

letter:/$ WifiPing 192.168.143.149
ping 192.168.143.149
GetCompleteATReply done
[reply result: +136

OK]
ping [192.168.143.149] ok
Device wifi ping success.
Return: 0, 0x00000000

letter:/$
```

Please support MobaXterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.net>

## Adapter发送数据

在连接上WIFI后，就可以用WIFI向外界发送数据了。过程如下：

先根据所使用的通讯协议，调用AdapterDeviceFindByName (ADAPTER\_WIFI\_NAME) 返回一个WIFI的 adapter，ADAPTER\_WIFI\_NAME这个宏是在APP\_Framework/Framework/connection/adapter.h文件中定义的。

然后设置所使用的协议，此处是TCP协议。设置后调用AdapterDeviceConnect进行TCP连接，连接成功即可使用AdapterDeviceSend向目标ip&端口发送数据了，发送的数据是字节流，应用层需要自行编写，在下文，我们会使用这个编写MQTT客户端的代码。

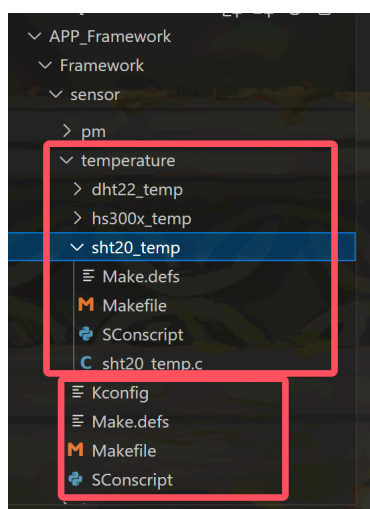
```
g_mqtt_adapter = AdapterDeviceFindByName(ADAPTER_WIFI_NAME);
g_mqtt_adapter->socket.protocol = SOCKET_PROTOCOL_TCP;
ret = AdapterDeviceConnect(g_mqtt_adapter, CLIENT, mqtt_server_addr->ipv4,
mqtt_server_addr->port, IPV4);
AdapterDeviceSend(g_mqtt_adapter, conn_buf, conn_len);
```

## 全面感知——传感器驱动开发

在XIUOS中，设计了一个全面感知的框架，采用了以物理量为中心的抽象方式，在开发时只需要考虑物理量的种类，无需把重点放在实际采集物理量的传感器设备，传感器设备的硬件细节已经处理好，只用知道传感器使用的什么通讯协议。所以我们在开发一个新传感器驱动时，不需要考虑硬件细节，只需要根据传感器的datasheet进行应用层的开发即可，这大大减少了传感器驱动开发的工作量。

以下是实际开发的标准过程：

### 新建传感器文件夹



目录结构如图所示，在我们开发新的传感器驱动时，应在APP\_Framework/sensor目录下开发，sensor下的每个目录对应一个抽象出来的物理量，如temperature，再下一级的目录是对应着传感器设备，因为每个物理量能够对应多个传感器设备，这个就是自己按需开发了。

在我们设计目录结构时，如果已经有物理量对应的目录了，就在对应目录下新建一个传感器的目录，然后参照图示的目录，创建所需的文件，这些文件可以从其他目录那复制过来，然后根据现在的传感器来修改对应的参数和宏。

### 传感器驱动实现

首先如上面目录所示，创建一个c文件，命名规范如图所示，通常是传感器型号+物理量，然后根据其他已有驱动的示例，再结合传感器的datasheet进行针对性开发，代码大纲及解释如下所示。

```
#include <sensor.h>

#define SHT20_CMD_SOFT_RESET      0xFE    //sht20传感器特有的指令
#define SHT20_CMD_TEMP_HOLD      0xE3

static struct SensorDevice sht20;          //每个驱动文件都要有的变量，是传感器设备的抽象
```

```

static struct SensorProductInfo info = { //传感器的产品信息，用于初始化传感器并
注册到XIUOS操作系统中
    (SENSOR_ABILITY_HUMI | SENSOR_ABILITY_TEMP),
    "Sensirion",
    "SHT20",
};

/**
 * @description: 打开SHT20传感器设备
 * @param sdev - 传感器设备指针
 * @return 成功: 0 错误: -1
 */
static int SensorDeviceOpen(struct SensorDevice *sdev) { //抽象出的对外的传感器操
作接口，可以在应用层调用，用于初始化传感器。以下两个函数同理
    int result;
    uint16_t i2c_dev_addr = SENSOR_DEVICE_SHT20_I2C_ADDR;

    sdev->fd = PrivOpen(SENSOR_DEVICE_SHT20_DEV, O_RDWR);
    if (sdev->fd < 0) {
        printf("open %s error\n", SENSOR_DEVICE_SHT20_DEV);
        return -1;
    }

    struct PrivIoctlCfg ioctl_cfg;
    ioctl_cfg.ioctl_driver_type = I2C_TYPE;
    ioctl_cfg.args = &i2c_dev_addr;
    result = PrivIoctl(sdev->fd, OPE_INT, &ioctl_cfg);

    return result;
}

/**
 * @description: 读取传感器数据
 * @param sdev - 传感器设备指针
 * @param len - 读取数据长度
 * @return 成功: 0, 失败: -1
 */
static int SensorDeviceRead(struct SensorDevice *sdev, size_t len) {
    if (PrivRead(sdev->fd, sdev->buffer, len) < 0)
        return -1;
    return 0;
}

/**
 * @description: 写入传感器命令
 * @param sdev - 传感器设备指针
 * @param buf - 写入数据缓冲区
 * @param len - 写入数据长度
 * @return 成功: 0, 失败: -1
 */
static int SensorDeviceWrite(struct SensorDevice *sdev, const void *buf, size_t
len) {
    if (Privwrite(sdev->fd, buf, len) < 0)
        return -1;
    return 0;
}

```

```

static struct SensorDone done = {    //用于将抽象的接口注册到XIUOS中
    SensorDeviceOpen,
    NULL,
    SensorDeviceRead,
    SensorDeviceWrite,
    NULL,
};

/**
 * @description: 初始化SHT20传感器并注册
 */
static void SensorDeviceSht20Init(void) {
    sht20.name = SENSOR_DEVICE_SHT20;
    sht20.info = &info;
    sht20.done = &done;
    sht20.status = SENSOR_DEVICE_PASSIVE;

    SensorDeviceRegister(&sht20);
}

static struct SensorQuantity sht20_temperature;    //传感器物理量的抽象，每个驱动文件
都要有

/**
 * @description: 温度信号转换为实际温度值
 * @param raw_data - 原始温度数据
 * @return 温度值( $^{\circ}\text{C} \times 10$ )
 */
static float Sht20ConvertTemperature(uint16_t raw_data) {
    // 清除状态位(最后两位)
    raw_data &= 0xFFFC;

    // 根据数据手册公式转换:  $T = -46.85 + 175.72 \times S_T / 2^{16}$ 
    float temperature = -46.85 + 175.72 * (raw_data / 65536.0);

    return temperature;
}

/**
 * @description: 读取SHT20温度值
 * @param quant - 传感器量指针
 * @return 温度值( $^{\circ}\text{C} \times 10$ )
 */
static int32_t ReadTemperature(struct SensorQuantity *quant) {
    if (!quant)
        return -1;

    uint8_t temp_cmd = SHT20_CMD_TEMP_HOLD;
    float result;

    if (quant->sdev->done->read != NULL && quant->sdev->done->write != NULL) {
        if (quant->sdev->status == SENSOR_DEVICE_PASSIVE) {
            // 发送温度测量命令
            if (quant->sdev->done->write(quant->sdev, &temp_cmd, 1) < 0) {
                printf("Send temperature command failed\n");
            }
        }
    }
}

```

```

        return -1;
    }

    // 等待测量完成(最大66ms)
    PrivTaskDelay(70);

    // 读取3字节数据(2字节数据 + 1字节CRC)
    if (quant->sdev->done->read(quant->sdev, 3) == 0) {
        // 组合温度数据
        uint16_t raw_temp = (quant->sdev->buffer[0] << 8) | quant->sdev-
>buffer[1];

        // 转换温度值
        result = Sht20ConvertTemperature(raw_temp);

        // 返回温度值(放大10倍)
        return (int32_t)(result * 10);
    }
    else {
        printf("Please set passive mode.\n");
    }
}
else {
    printf("%s don't have read/write done.\n", quant->name);
}

return -1;
}

/**
 * @description: 初始化SHT20温度量程并注册
 * @return 0
 */
int Sht20TemperatureInit(void) { //初始化函数，用于在
APP_Framework/Framework/framework_init.c中的FrameworkInit进行调用
    SensorDeviceSht20Init();

    sht20_temperature.name = SENSOR_QUANTITY_SHT20_TEMPERATURE;
    sht20_temperature.type = SENSOR_QUANTITY_TEMP;
    sht20_temperature.value.decimal_places = 1;
    sht20_temperature.value.max_std = 850; // -40°C to +125°C
    sht20_temperature.value.min_std = -400; // -40.0°C
    sht20_temperature.value.last_value = SENSOR_QUANTITY_VALUE_ERROR;
    sht20_temperature.value.max_value = SENSOR_QUANTITY_VALUE_ERROR;
    sht20_temperature.value.min_value = SENSOR_QUANTITY_VALUE_ERROR;
    sht20_temperature.sdev = &sht20;
    sht20_temperature.ReadValue = ReadTemperature;

    SensorQuantityRegister(&sht20_temperature);

    return 0;
}

```

在开发完驱动后，需要到APP\_Framework/Framework/framework\_init.c中的FrameworkInit进行注册。



首先先extern对应的初始化函数，如下图所示。

```
APP_Framework > Framework > C framework_init.c > Sht20TemperatureInit(void)
28
29 extern int Bmp180AltitudeInit(void);
30 extern int D124VoiceInit(void);
31 extern int Hs300xTemperatureInit(void);
32 extern int Hs300xHumidityInit(void);
33 extern int Ps5308Pm1_0Init(void);
34 extern int Ps5308Pm2_5Init(void);
35 extern int Ps5308Pm10Init(void);
36 extern int Zg09Co2Init(void);
37 extern int G8sCo2Init(void);
38 extern int As830Ch4Init(void);
39 extern int Tb600bIaq10IaqInit(void);
40 extern int Tb600bTvoc10TvocInit(void);
41 extern int Tb600bWqHcho10sInit(void);
42 extern int QsFxWindDirectionInit(void);
43 extern int QsFsWindSpeedInit(void);
44 extern int HcSr501Init(void);
45 extern int Dht22TemperatureInit(void);
46 extern int Sht20TemperatureInit(void);
47 extern int Sht20HumidityInit(void);
48
49 extern int lv_port_init(void);
50
```

然后参照这里的格式进行函数指针的连接

```
APP_Framework > Framework > C framework_init.c > Sht20TemperatureInit(void)
90 {
91     #ifdef SENSOR_DEVICE_HS300X
92         { "hs300x_temperature", Hs300xTemperatureInit },
93     #endif
94     #ifdef SENSOR_QUANTITY_HS300X_HUMIDITY
95         { "hs300x_humidity", Hs300xHumidityInit },
96     #endif
97     #endif
98
99     #ifdef SENSOR_DEVICE_SHT20
100     #ifdef SENSOR_QUANTITY_SHT20_TEMPERATURE
101         { "sht20_temperature", Sht20TemperatureInit },
102     #endif
103     #ifdef SENSOR_QUANTITY_SHT20_HUMIDITY
104         { "sht20_humidity", Sht20HumidityInit },
105     #endif
106     #endif
107
108     #ifdef SENSOR_DEVICE_DHT22
109     #ifdef SENSOR_QUANTITY_DHT22_TEMPERATURE
110         { "dht22_temperature", Dht22TemperatureInit },
111     #endif
112     #endif
113 }
114
```

之后就可以编写应用层的程序进行传感器数据的读取。如在  
APP\_Framework/Applications/sensor\_app/temperature\_sht20.c进行编写。

代码如下所示

```
#include <sensor.h>

/**
 * @description: Read a temperature
 * @return 0
 */
void TempSht20(void)    //用于测试
{
    int i = 0;
```

```

int32_t temperature;
struct SensorQuantity *temp =
SensorQuantityFind(SENSOR_QUANTITY_SHT20_TEMPERATURE, SENSOR_QUANTITY_TEMP);
SensorQuantityOpen(temp);

for (i = 0; i < 10; i++) {
    temperature = SensorQuantityReadValue(temp);
    if (temperature > 0)
        printf("Temperature : %d.%d C\n", temperature/10, temperature%10);
    else
        printf("Temperature : %d.%d C\n", temperature/10, -temperature%10);

    PrivTaskDelay(500);
}

SensorQuantityClose(temp);
}

struct SensorQuantity *GetTempQuantity(void)           //向外提供一个物理量指针，可以由外
部进行数据读取
{
    struct SensorQuantity *temp =
SensorQuantityFind(SENSOR_QUANTITY_SHT20_TEMPERATURE, SENSOR_QUANTITY_TEMP);
    SensorQuantityOpen(temp);
    return temp;
}

```

## Makefile和Kconfig的编写注意事项

### 传感器驱动目录的编写

举例来说，首先对于APP\_Framework/Framework/sensor/temperature/sht20\_temp这种传感器的驱动目录下的Make.defs，圈出来的这几处是需要修改的，CONFIG\_SENSOR\_SHT20\_TEMPERATURE是一个宏，在Kconfig配置启用了SENSOR\_SHT20\_TEMPERATURE之后就会被定义，所以这个宏和APP\_Framework/Framework/sensor/temperature目录下的Kconfig文件有关。

```

APP_Framework > Framework > sensor > temperature > sht20_temp > Make.defs
1  #####
2  # APP_Framework/Framework/sensor/temperature/sht20_temp/Make.defs
3  #####
4  ifneq $(CONFIG_SENSOR_SHT20_TEMPERATURE),
5  CONFIGURED_APPS += $(APPDIR)/../../../../APP_Framework/Framework/sensor/temperature/sht20_temp
6  endif
7

```

PS：关于Kconfig文件

这是一个用于简易配置宏的文件，Kfrontend是一个可视化界面配置宏的工具，可以通过这个工具选择是否启用宏，Kconfig就是这个工具的配置文件，可以自定义宏的值和分支。

```

APP_Framework > Framework > sensor > temperature > sht20_temp > Makefile
1 include $(KERNEL_ROOT)/.config
2 ifeq ($(CONFIG_ADD_NUTTX_FEATURES),y)
3     include $(APPDIR)/Make.defs
4     CSRCS += sht20_temp.c
5     include $(APPDIR)/Application.mk
6
7 endif
8
9 ifeq ($(CONFIG_ADD_XIZI_FEATURES),y)
10    SRC_FILES := sht20_temp.c
11    include $(KERNEL_ROOT)/compiler.mk
12 endif
13
APP_Framework > Framework > sensor > temperature > Makefile
1 ifeq ($(CONFIG_SENSOR_HS300X_TEMPERATURE),y)
2     SRC_DIR += hs300x_temp
3 endif
4
5 ifeq ($(CONFIG_SENSOR_DHT22_TEMPERATURE),y)
6     SRC_DIR += dht22_temp
7 endif
8
9 ifeq ($(CONFIG_SENSOR_SHT20_TEMPERATURE),y)
10    SRC_DIR += sht20_temp
11 endif
12
13
14 include $(KERNEL_ROOT)/compiler.mk
15

```

不同目录下的Makefile的修改如上图所示，目录的路径看图可知。

对于Kconfig文件来说，语法可以自行ai，我们需要做的就是根据传感器所需的各个值，定义相应的宏，然后使用Kfrontend简易配置宏的值，供驱动使用，这里解释一下各个宏。

SENSOR\_SHT20\_TEMPERATURE——定义是否启用这个物理量，此处对应的就是Make.defs中的宏

SENSOR\_DEVICE\_SHT20——这个宏是有值的，默认为sht20\_1，用处就是在sht20\_temp.c文件中使用，c文件中多个宏都是Kconfig中定义的

SENSOR\_QUANTITY\_SHT20\_TEMPERATURE——同上解释，其他宏同理

**注意！**

注意缩进是否正确！

```

config SENSOR_SHT20_TEMPERATURE
    bool "Using SHT20 for temperature"
    default n

    if SENSOR_SHT20_TEMPERATURE
        config SENSOR_DEVICE_SHT20
            string "SHT20 sensor name"
            default "sht20_1"

        config SENSOR_QUANTITY_SHT20_TEMPERATURE
            string "sht20 quantity name"
            default "temperature_1"

        if ADD_XIZI_FEATURES
            config SENSOR_DEVICE_SHT20_DEV
                string "SHT20 device name"
                default "/dev/i2c1_dev0"

            config SENSOR_DEVICE_SHT20_I2C_ADDR
                hex "SHT20 device i2c address"

```

```

                                default 0x40

endif

if ADD_NUTTX_FEATURES
    config SENSOR_DEVICE_SHT20_DEV
        string "SHT20 device name"
        default "/dev/i2c1_dev0"

    config SENSOR_DEVICE_SHT20_I2C_ADDR
        hex "SHT20 device i2c address"
        default 0x40

endif

if ADD_RTTHREAD_FEATURES
    config SENSOR_DEVICE_SHT20_DEV
        string "SHT20 device name"
        default "/dev/i2c1"
    config SENSOR_DEVICE_SHT20_I2C_ADDR
        hex "SHT20 device i2c address"
        default 0x40

endif

endif
endif

```

此外，如果要开发新的物理量，需要在APP\_Framework/Framework/sensor目录下的Makefile和Kconfig文件中添加对应的选项和宏，在APP\_Framework/Framework/sensor/sensor.h中也要进行物理量的定义，这个可以自行查看。

### 应用层目录的编写

APP\_Framework/Applications即应用层的程序，这里以APP\_Framework/Applications/sensor\_app目录举例子。

我们开发完驱动后，如果需要开发应用层的程序，可以把程序写到这个目录下供main.c调用，然后为了main.c能够查找到对应的文件，我们需要配置好Makefile。这里是

APP\_Framework/Applications/Makefile的截图，可以看到sensor\_app已经有配置了，就可以不用管，APP\_Framework/Applications/Kconfig同理

```

APP_Framework > Applications > M Makefile
10  ifeq ($(CONFIG_ADD_XIZI_FEATURES),y)
22  ifeq ($(CONFIG_APPLICATION_OTA),y)
23  SRC_DIR += ota
24  endif
25
26  ifeq ($(CONFIG_APPLICATION_SENSOR),y)
27  SRC_DIR += sensor_app
28  endif
29

```

```

11         default 85 if KTASK_PRIORITY_256
12     endmenu
13
14     source "$APP_DIR/Applications/ota/Kconfig"
15
16     source "$APP_DIR/Applications/app_test/Kconfig"
17     source "$APP_DIR/Applications/connection_app/Kconfig"
18     source "$APP_DIR/Applications/control_app/Kconfig"
19     source "$APP_DIR/Applications/knowning_app/Kconfig"
20     source "$APP_DIR/Applications/sensor_app/Kconfig"
21     source "$APP_DIR/Applications/embedded_database_app/Kconfig"
22     source "$APP_DIR/Applications/webnet/Kconfig"
23     source "$APP_DIR/Applications/webserver/Kconfig"
24     source "$APP_DIR/Applications/mutex/Kconfig"

```

接下来看到APP\_Framework/Applications/sensor\_app下的Makefile，当我们在APP\_Framework/Applications/sensor\_app创建一个新文件后，需要在这配置一下，让Makefile能够找到文件并编译，这里有分支判断的处理，这个宏是Kconfig文件内的宏。

```

APP_Framework > Applications > sensor_app > M Makefile
71 ifeq ($(CONFIG_ADD_XIZI_FEATURES),y)
120 endif
121
122 ifeq ($(CONFIG_APPLICATION_SENSOR_TEMPERATURE_HS300X), y)
123     SRC_FILES += temperature_hs300x.c
124 endif
125
126 ifeq ($(CONFIG_APPLICATION_SENSOR_TEMPERATURE_DHT22), y)
127     SRC_FILES += temperature_dht22.c
128 endif
129
130 ifeq ($(CONFIG_APPLICATION_SENSOR_TEMPERATURE_SHT20), y)
131     SRC_FILES += temperature_sht20.c
132 endif
133
134 ifeq ($(CONFIG_APPLICATION_SENSOR_MOTION_HCSR501), y)
135     SRC_FILES += motion_hcsr501.c
136 endif
137
138 ifeq ($(CONFIG_APPLICATION_SENSOR_WINDDIRECTION_QS_FX), y)

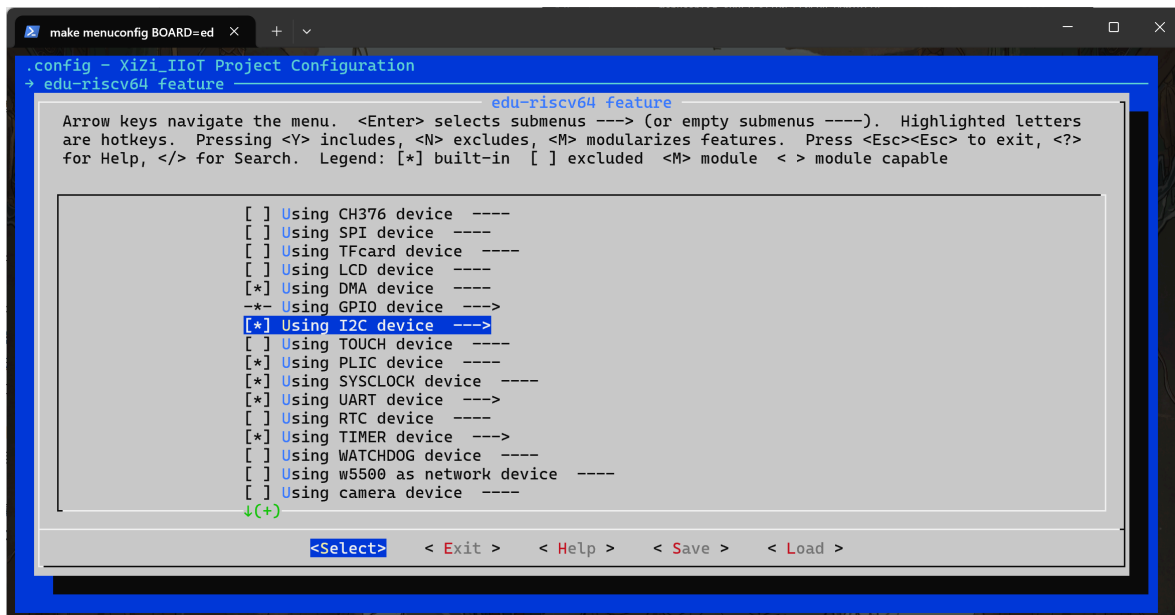
```

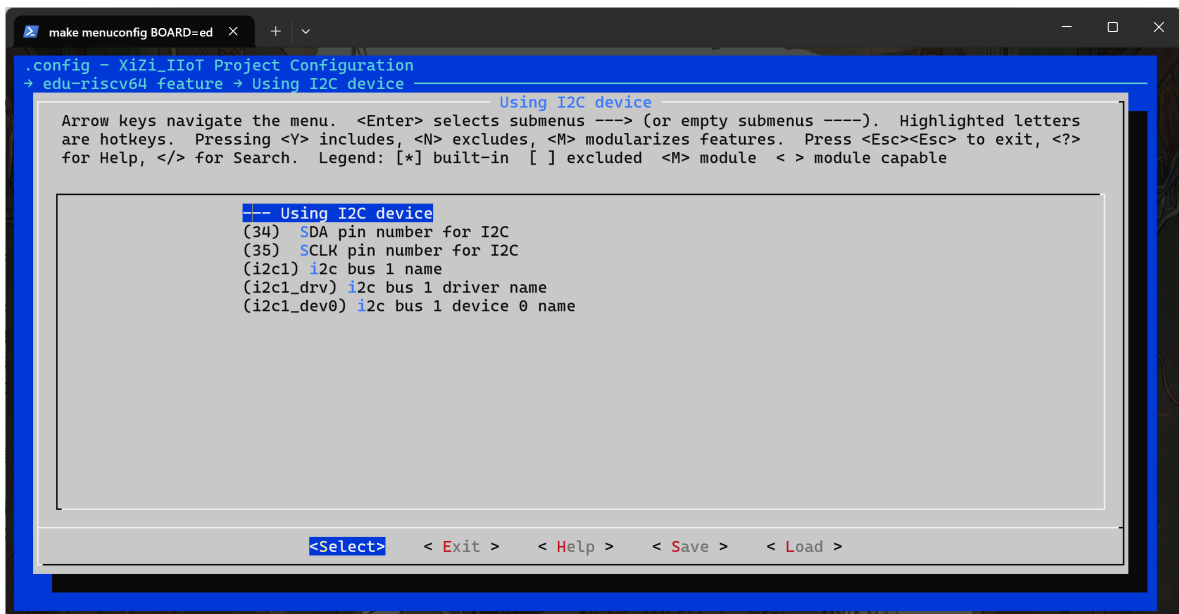
如下所示，在Kconfig中添加如下配置，当在Kfrontend中勾选了"using sensor SHT20 apps"后，会定义一个名称为APPLICATION\_SENSOR\_TEMPERATURE\_SHT20的宏，同时也会定义一个CONFIG\_APPLICATION\_SENSOR\_TEMPERATURE\_SHT20宏，这样在Makefile里就会把对应的文件加入编译。

```
APP_Framework > Applications > sensor_app > Kconfig
1  menu "sensor app"
3  menuconfig APPLICATION_SENSOR
7  if APPLICATION_SENSOR
102 menuconfig APPLICATION_SENSOR_TEMPERATURE
116 if APPLICATION_SENSOR_TEMPERATURE
120
121     if ADD_NUTTX_FEATURES
122     endif
123
124 endif
125
126     if APPLICATION_SENSOR_TEMPERATURE
127     config APPLICATION_SENSOR_TEMPERATURE_SHT20
128     bool "Using sensor SHT20 apps"
129     default n
130
131     if ADD_NUTTX_FEATURES
132     endif
133
134     endif
135
136 menuconfig APPLICATION_SENSOR_HUMIDITY
137 bool "Using sensor humidity apps"
138 default n
139
```

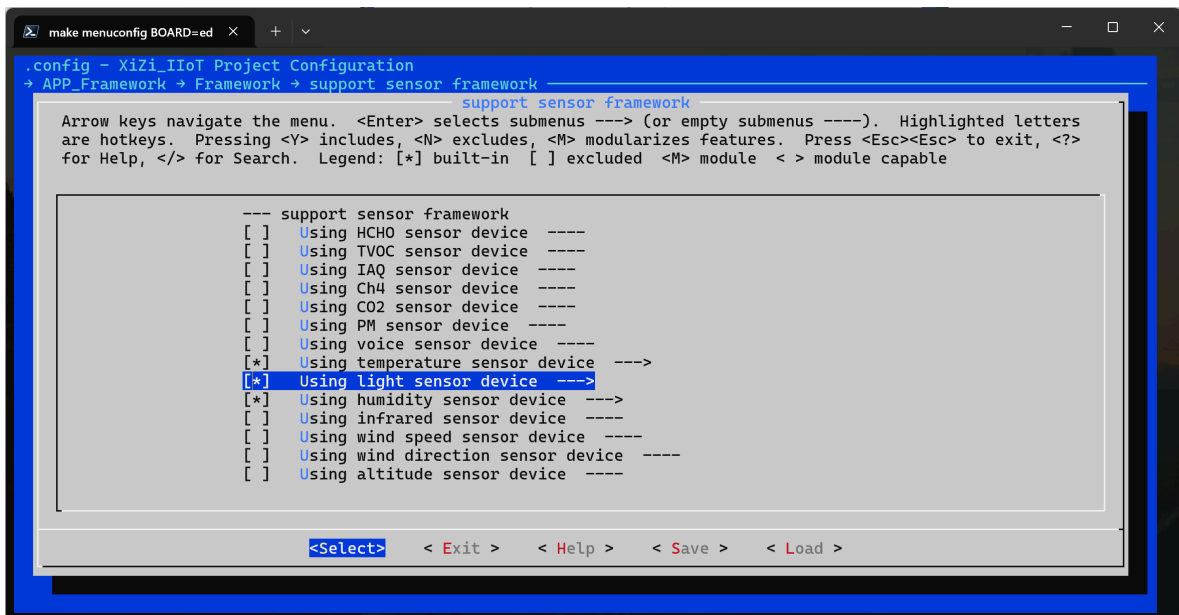
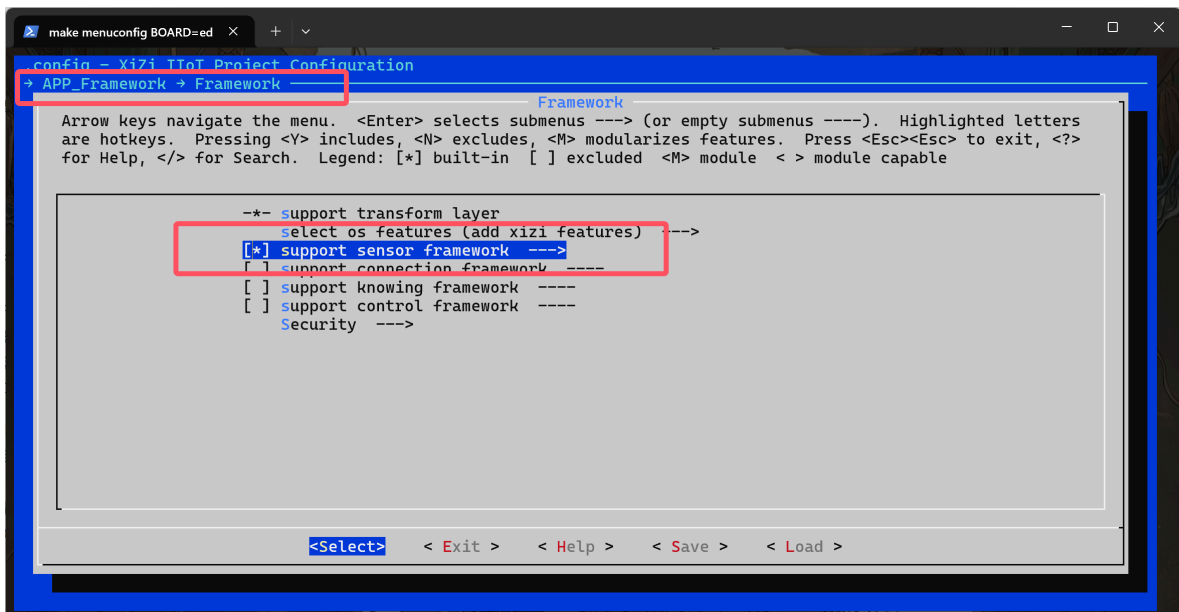
## 配置过程

首先勾选I2C，对于edu-riscv平台，两个数据引脚分别是34和35

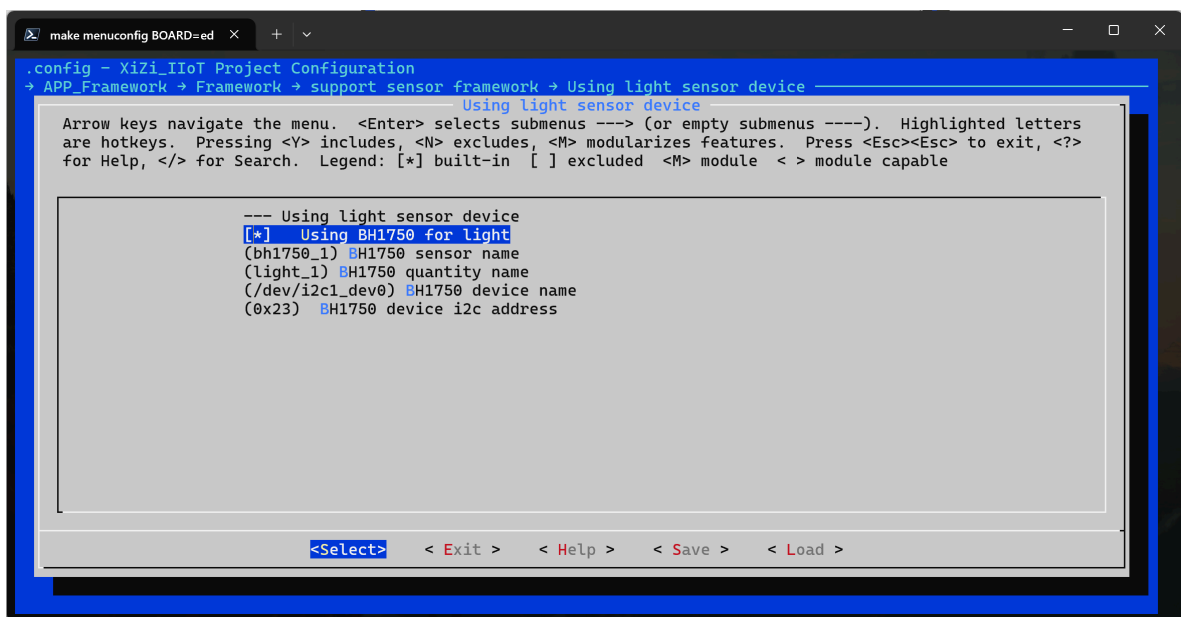
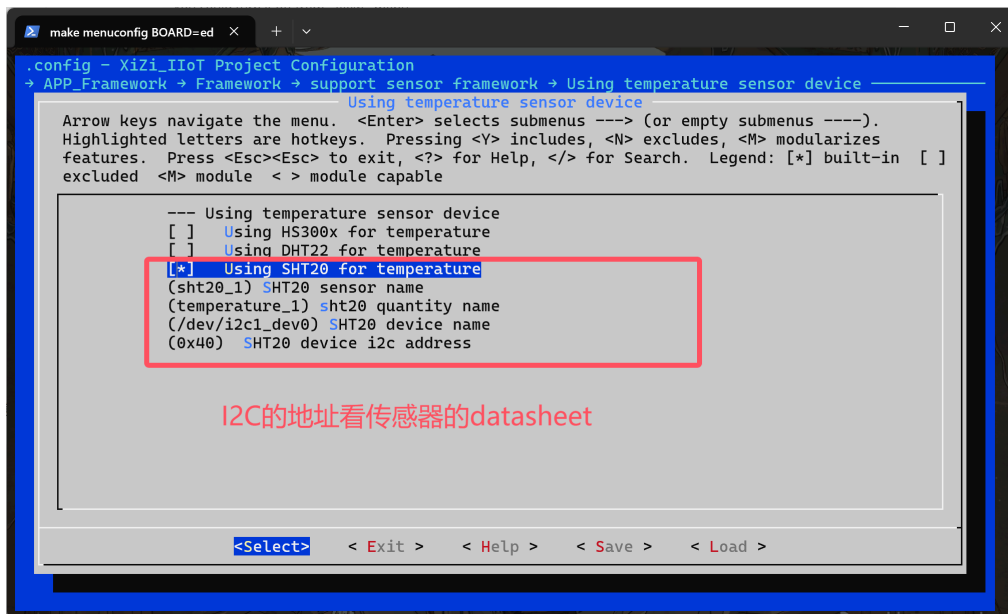




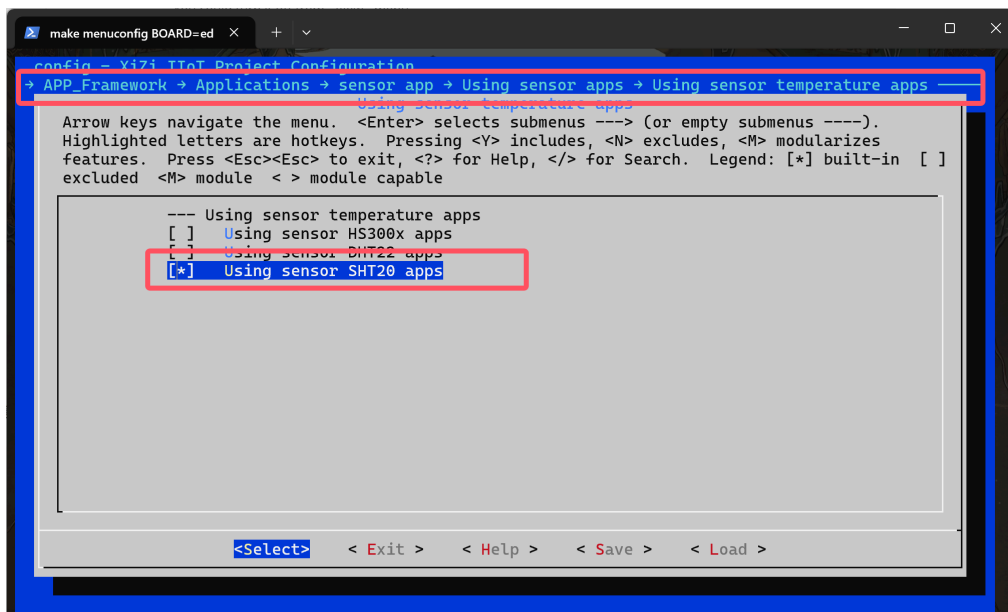
然后去勾选相应的传感器驱动







然后应用层的文件也进行选择，供main.c使用，其他应用层的代码可以写在Applications的其他目录下，按照上述过程进行配置就可让main.c使用了。



之后疯狂esc然后按Y保存配置



## 传感器开发情况

在本次项目中，我们小组探索并开发HCSR501红外传感器、DHT22温湿度传感器、SHT20温湿度传感器、BH1750光照传感器。

在探索过程中，由于自身能力问题和对嵌入式操作系统底层的不熟悉，对HCSR501和DHT22的探索以失败告终，但成功开发了SHT20和BH1750这两个传感器，需要使用多个传感器时，将传感器相同引脚并联，接在板端。以下是驱动实现。

### HCSR501

HCSR501是基于GPIO，无通讯协议的传感器，所以只能根据它的高低电平变化来设计中断处理函数来处理事件，在测试过程中，我们发现中断处理函数总是会延迟触发，初步判断是XIUOS缓冲区和调度的问题，出于实际情况考虑并未深入探索，放弃开发此传感器。

### DHT22

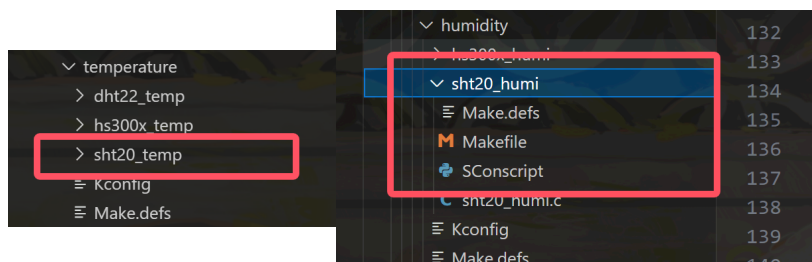
DHT22也是基于GPIO，无通讯协议的传感器，读取数据需要根据说明书里的时序进行解析。

由于未接触过这种底层的微秒级别的时序，也没开发过类似的应用，XIUOS提供的库并未有微秒级别的获取时间方法和delay方法，所以在开发了三天后也只能放弃后续开发。

### SHT20

SHT20是使用了I2C协议进行通讯的传感器，在XIUOS中已经对I2C协议进行封装，所以使用对应的接口进行开发非常的简单，能够很容易的读取数据，最后成功读取了传感器数据。

因为SHT20是温湿度传感器，而XIUOS抽象的粒度是一个物理量，所以分为两个部分进行开发，如下图所示。



在上文传感器驱动开发流程中已经给了温度的驱动代码，以下是湿度的驱动代码。

```
#include <sensor.h>

#define SHT20_CMD_HUMI_HOLD          0xE5
#define SHT20_CMD_HUMI_NOHOLD       0xF5

static struct SensorDevice sht20;

static struct SensorProductInfo humidity_info = {
    (SENSOR_ABILITY_HUMI | SENSOR_ABILITY_TEMP),
    "Sensirion",
    "SHT20",
};
```

```

/**
 * @description: 打开SHT20湿度传感器设备
 * @param sdev - 传感器设备指针
 * @return 成功: 0 错误: -1
 */
static int HumidityDeviceOpen(struct SensorDevice *sdev) {
    int result;
    uint16_t i2c_dev_addr = SENSOR_DEVICE_SHT20_I2C_ADDR;

    sdev->fd = PrivOpen(SENSOR_DEVICE_SHT20_DEV, O_RDWR);
    if (sdev->fd < 0) {
        printf("open %s error\n", SENSOR_DEVICE_SHT20_DEV);
        return -1;
    }

    struct PrivIoctlCfg ioctl_cfg;
    ioctl_cfg.ioctl_driver_type = I2C_TYPE;
    ioctl_cfg.args = &i2c_dev_addr;
    result = PrivIoctl(sdev->fd, OPE_INT, &ioctl_cfg);

    return result;
}

/**
 * @description: 读取湿度传感器数据
 * @param sdev - 传感器设备指针
 * @param len - 读取数据长度
 * @return 成功: 0, 失败: -1
 */
static int HumidityDeviceRead(struct SensorDevice *sdev, size_t len) {
    if (PrivRead(sdev->fd, sdev->buffer, len) < 0)
        return -1;
    return 0;
}

/**
 * @description: 写入湿度传感器命令
 * @param sdev - 传感器设备指针
 * @param buf - 写入数据缓冲区
 * @param len - 写入数据长度
 * @return 成功: 0, 失败: -1
 */
static int HumidityDeviceWrite(struct SensorDevice *sdev, const void *buf, size_t len) {
    if (PrivWrite(sdev->fd, buf, len) < 0)
        return -1;
    return 0;
}

static struct SensorDone humidity_done = {
    HumidityDeviceOpen,
    NULL,
    HumidityDeviceRead,
    HumidityDeviceWrite,
    NULL,
};

```

```

/**
 * @description: 初始化SHT20湿度传感器并注册
 */
static void SensorDeviceSht20HumidityInit(void) {
    sht20.name = SENSOR_DEVICE_SHT20 ;
    sht20.info = &humidity_info;
    sht20.done = &humidity_done;
    sht20.status = SENSOR_DEVICE_PASSIVE;

    SensorDeviceRegister(&sht20);
}

static struct SensorQuantity sht20_humidity;

/**
 * @description: 湿度信号转换为实际湿度值
 * @param raw_data - 原始湿度数据
 * @return 湿度值(%RH × 10)
 */
static float Sht20ConvertHumidity(uint16_t raw_data) {
    // 清除状态位(最后两位)
    raw_data &= 0xFFFC;

    // 根据数据手册公式转换:  $RH = -6 + 125 \times S_{RH} / 2^{16}$ 
    float humidity = -6.0 + 125.0 * (raw_data / 65536.0);

    // 限制湿度范围在0-100%RH之间
    if (humidity < 0.0) humidity = 0.0;
    if (humidity > 100.0) humidity = 100.0;

    return humidity;
}

/**
 * @description: 读取SHT20湿度值
 * @param quant - 传感器量指针
 * @return 湿度值(%RH × 10)
 */
static int32_t ReadHumidity(struct SensorQuantity *quant) {
    if (!quant)
        return -1;

    uint8_t humi_cmd = SHT20_CMD_HUMI_NOHOLD;
    float result;

    if (quant->sdev->done->read != NULL && quant->sdev->done->write != NULL) {
        if (quant->sdev->status == SENSOR_DEVICE_PASSIVE) {
            // 发送湿度测量命令
            if (quant->sdev->done->write(quant->sdev, &humi_cmd, 1) < 0) {
                printf("Send humidity command failed\n");
                return -1;
            }

            // 等待测量完成(最大22ms for 12bit分辨率)
            PrivTaskDelay(30);
        }
    }
}

```

```

        // 读取3字节数据(2字节数据 + 1字节CRC)
        if (quant->sdev->done->read(quant->sdev, 3) == 0) {
            // 组合湿度数据
            uint16_t raw_humi = (quant->sdev->buffer[0] << 8) | quant->sdev->buffer[1];

            // 转换湿度值
            result = Sht20ConvertHumidity(raw_humi);

            // printf("SHT20 Humidity - Raw: 0x%04X, Converted: %.1f%%RH\n",
            //       raw_humi, result);

            // 返回湿度值(放大10倍)
            return (int32_t)(result * 10);
        } else {
            printf("Read humidity data failed\n");
        }
    } else {
        printf("Please set passive mode for humidity reading.\n");
    }
} else {
    printf("%s don't have read/write done.\n", quant->name);
}

return -1;
}

/**
 * @description: 带温度补偿的湿度读取（更精确）
 * @param quant - 传感器量指针
 * @param temperature - 当前温度值(°C × 10)
 * @return 补偿后的湿度值(%RH × 10)
 */
static int32_t ReadHumiditywithCompensation(struct SensorQuantity *quant, int32_t temperature) {
    if (!quant)
        return -1;

    // 先读取原始湿度
    int32_t raw_humidity = ReadHumidity(quant);
    if (raw_humidity == -1) {
        return -1;
    }

    float humidity = raw_humidity / 10.0; // 转换为%RH
    float temp_c = temperature / 10.0;    // 转换为°C

    // 根据数据手册进行温度补偿
    // 在高温高湿环境下进行补偿
    if (temp_c > 25.0 && humidity > 60.0) {
        // 简单的线性补偿（实际应用中应根据数据手册使用更精确的公式）
        float compensation = (temp_c - 25.0) * 0.1;
        humidity = humidity - compensation;

        printf("Applied temperature compensation: -%.1f%%RH\n", compensation);
    }
}

```

```

    }

    // 确保湿度在有效范围内
    if (humidity < 0.0) humidity = 0.0;
    if (humidity > 100.0) humidity = 100.0;

    return (int32_t)(humidity * 10);
}

/**
 * @description: 读取湿度和温度的综合函数
 * @param quant - 传感器量指针
 * @param temperature - 输出温度值(°C × 10)
 * @param humidity - 输出湿度值(%RH × 10)
 * @return 成功: 0, 失败: -1
 */
static int ReadHumidityAndTemperature(struct SensorQuantity *quant,
                                      int32_t *temperature, int32_t *humidity) {
    if (!quant || !temperature || !humidity)
        return -1;

    // 这里可以实现同时读取温湿度的逻辑
    // 根据SHT20数据手册, 可以优化读取顺序

    *temperature = -1;
    *humidity = -1;

    // 实际应用中可能需要更复杂的读取策略
    return -1; // 暂不实现
}

/**
 * @description: 初始化SHT20湿度量程并注册
 * @return 成功: 0, 失败: -1
 */
int Sht20HumidityInit(void) {
    SensorDeviceSht20HumidityInit();

    sht20_humidity.name = SENSOR_QUANTITY_SHT20_HUMIDITY;
    sht20_humidity.type = SENSOR_QUANTITY_HUMI;
    sht20_humidity.value.decimal_places = 1;
    sht20_humidity.value.max_std = 1000; // 0% to 100% RH
    sht20_humidity.value.min_std = 0; // 0.0% RH
    sht20_humidity.value.last_value = SENSOR_QUANTITY_VALUE_ERROR;
    sht20_humidity.value.max_value = SENSOR_QUANTITY_VALUE_ERROR;
    sht20_humidity.value.min_value = SENSOR_QUANTITY_VALUE_ERROR;
    sht20_humidity.sdev = &sht20;
    sht20_humidity.ReadValue = ReadHumidity;

    SensorQuantityRegister(&sht20_humidity);
    return 0;
}

```

## BH1750

BH1750也是使用I2C协议进行通讯的光照传感器，开发过程同SHT20，驱动代码如下所示。

```
#include <sensor.h>

#define BH1750_CMD_POWER_ON      0x01
#define BH1750_CMD_RESET        0x07
#define BH1750_CMD_MEASUREMENT  0x20

static struct SensorDevice bh1750;

static struct SensorProductInfo info = {
    SENSOR_ABILITY_LIGHT, // 光照传感器能力
    "ROHM",               // 制造商
    "BH1750",             // 型号
};

/**
 * @description: 打开BH1750传感器设备
 * @param sdev - 传感器设备指针
 * @return 成功: 0 错误: -1
 */
static int SensorDeviceOpen(struct SensorDevice *sdev) {
    int result;
    uint16_t i2c_dev_addr = SENSOR_DEVICE_BH1750_I2C_ADDR;

    sdev->fd = PrivOpen(SENSOR_DEVICE_BH1750_DEV, O_RDWR);
    if (sdev->fd < 0) {
        printf("open %s error\n", SENSOR_DEVICE_BH1750_DEV);
        return -1;
    }

#ifdef ADD_XIZI_FEATURES
    struct PrivIoctlCfg ioctl_cfg;
    ioctl_cfg.ioctl_driver_type = I2C_TYPE;
    ioctl_cfg.args = &i2c_dev_addr;
    result = PrivIoctl(sdev->fd, OPE_INT, &ioctl_cfg);
#else
    result = 0; // 非XIZI环境可能需要其他初始化方式
#endif

    return result;
}

/**
 * @description: 关闭BH1750传感器设备
 * @param sdev - 传感器设备指针
 * @return 成功: 0
 */
static int SensorDeviceClose(struct SensorDevice *sdev) {
    if (sdev->fd >= 0) {
        PrivClose(sdev->fd);
        sdev->fd = -1;
    }

    return 0;
}
```

```

}

/**
 * @description: 读取传感器数据
 * @param sdev - 传感器设备指针
 * @param len - 读取数据长度
 * @return 成功: 0, 失败: -1
 */
static int SensorDeviceRead(struct SensorDevice *sdev, size_t len) {
    if (PrivRead(sdev->fd, sdev->buffer, len) < 0) {
        printf("BH1750 read error\n");
        return -1;
    }
    return 0;
}

/**
 * @description: 写入传感器命令
 * @param sdev - 传感器设备指针
 * @param buf - 写入数据缓冲区
 * @param len - 写入数据长度
 * @return 成功: 0, 失败: -1
 */
static int SensorDeviceWrite(struct SensorDevice *sdev, const void *buf, size_t len) {
    if (PrivWrite(sdev->fd, buf, len) < 0) {
        printf("BH1750 write error\n");
        return -1;
    }
    return 0;
}

/**
 * @description: 复位BH1750传感器
 * @param sdev - 传感器设备指针
 * @return 成功: 0, 失败: -1
 */
static int SensorDeviceReset(struct SensorDevice *sdev) {
    uint8_t reset_cmd = BH1750_CMD_RESET;

    if (sdev->done->write(sdev, &reset_cmd, 1) < 0) {
        printf("BH1750 reset failed\n");
        return -1;
    }

    PrivTaskDelay(5); // 等待复位完成
    return 0;
}

/**
 * @description: 上电BH1750传感器
 * @param sdev - 传感器设备指针
 * @return 成功: 0, 失败: -1
 */
static int SensorDevicePowerOn(struct SensorDevice *sdev) {
    uint8_t power_on_cmd = BH1750_CMD_POWER_ON;

```

```

        if (sdev->done->write(sdev, &power_on_cmd, 1) < 0) {
            printf("BH1750 power on failed\n");
            return -1;
        }

        PrivTaskDelay(5); // 等待上电完成
        return 0;
    }

static struct SensorDone done = {
    SensorDeviceOpen,
    SensorDeviceClose,
    SensorDeviceRead,
    SensorDeviceWrite,
    SensorDeviceReset,
};

/**
 * @description: 初始化BH1750传感器并注册
 */
static void SensorDeviceBh1750Init(void) {
    bh1750.name = SENSOR_DEVICE_BH1750;
    bh1750.info = &info;
    bh1750.done = &done;
    bh1750.status = SENSOR_DEVICE_PASSIVE;

    SensorDeviceRegister(&bh1750);
}

static struct SensorQuantity bh1750_light;

/**
 * @description: 光照强度信号转换为实际值
 * @param raw_data - 原始光照数据
 * @return 光照强度值(lx)
 */
static float Bh1750ConvertLight(uint16_t raw_data) {
    // BH1750转换公式: 光照强度(lx) = 原始值 / 1.2
    // 高分辨率模式下, 分辨率=1 lx/count
    float light_intensity = (float)raw_data / 1.2f;
    return light_intensity;
}

/**
 * @description: 读取BH1750光照强度值
 * @param quant - 传感器量指针
 * @return 光照强度值(lx × 10)
 */
static int32_t ReadLightIntensity(struct SensorQuantity *quant) {
    if (!quant) {
        return -1;
    }

    uint8_t measure_cmd = BH1750_CMD_MEASUREMENT;
    float result;

```



```

    if (quant->sdev->done->read != NULL && quant->sdev->done->write != NULL) {
        if (quant->sdev->status == SENSOR_DEVICE_PASSIVE) {
            // 发送测量命令
            if (quant->sdev->done->write(quant->sdev, &measure_cmd, 1) < 0) {
                printf("Send BH1750 measurement command failed\n");
                return -1;
            }

            // 等待测量完成（高分辨率模式最大180ms）
            PrivTaskDelay(200);

            // 读取2字节数据
            if (quant->sdev->done->read(quant->sdev, 2) == 0) {
                // 组合光照数据
                uint16_t raw_light = (quant->sdev->buffer[0] << 8) | quant->sdev->buffer[1];

                // 转换光照强度值
                result = Bh1750ConvertLight(raw_light);

                // 返回光照强度值(放大10倍保持精度)
                return (int32_t)(result * 10);
            }
        } else {
            printf("Please set passive mode.\n");
        }
    } else {
        printf("%s don't have read/write done.\n", quant->name);
    }

    return -1;
}

/**
 * @description: 初始化BH1750光照量程并注册
 * @return 0
 */
int Bh1750LightInit(void) {
    SensorDeviceBh1750Init();

    bh1750_light.name = SENSOR_QUANTITY_BH1750_LIGHT;
    bh1750_light.type = SENSOR_QUANTITY_LIGHT;
    bh1750_light.value.decimal_places = 1;           // 小数点后1位
    bh1750_light.value.max_std = 655350;            // 最大光照强度 65535 lx
    bh1750_light.value.min_std = 0;                  // 最小光照强度 0 lx
    bh1750_light.value.last_value = SENSOR_QUANTITY_VALUE_ERROR;
    bh1750_light.value.max_value = SENSOR_QUANTITY_VALUE_ERROR;
    bh1750_light.value.min_value = SENSOR_QUANTITY_VALUE_ERROR;
    bh1750_light.sdev = &bh1750;
    bh1750_light.ReadValue = ReadLightIntensity;

    SensorQuantityRegister(&bh1750_light);

    return 0;
}

```

## 多任务开发

在本系统的设计中，有多个传感器量的数据读取，以及板端与服务器端进行的数据交互，这就要求程序能够支持多线程的并发执行。

在Ubiquitous/XiZi\_IloT/kernel/include/user\_api.h这个文件中，提供了任务线程建立、锁、消息队列等功能的用户接口，开发程序时可以使用这些接口进行多任务的开发，当任务优先级相同时，各个线程会按照时间片轮转算法分配cpu。

首先把需要拆出的任务写成一个单独函数，然后进行任务创建，如下所示。

```
strncpy(temp_task.name, "temp_task", NAME_NUM_MAX - 1);
temp_task.func_entry = (void *)TemperatureTask;
temp_task.func_param = (void *)&temperature_task_run;
temp_task.stack_size = SENSOR_TASK_STACK_SIZE;
temp_task.prio = TEMPERATURE_TASK_PRIORITY;

temperature_task_id = UserTaskCreate(temp_task);
```

## 锁的开发

锁的用途主要是管理总线的临界区，使用了IO34&35的传感器或控制器都需要加上这个锁，防止缓冲区出错。

```
//APP_Framework/Applications/mutex/sensor_mutex.c

#include "sensor_mutex.h"
#include <stdio.h>

int32_t sht20_mutex = -1;

/**
 * @description: 初始化传感器互斥锁
 * @return 成功: 0, 失败: -1
 */
int SensorMutexInit(void)
{
    if (sht20_mutex == -1) {
        sht20_mutex = UserMutexCreate();
        if (sht20_mutex < 0) {
            printf(" Failed to create sensor mutex\n");
            return -1;
        }
        printf(" Sensor mutex created successfully (ID: %d)\n", sht20_mutex);
    }
    return 0;
}
```

```

/**
 * @description: 销毁传感器互斥锁
 */
void SensorMutexDeinit(void)
{
    if (sht20_mutex != -1) {
        UserMutexDelete(sht20_mutex);
        sht20_mutex = -1;
        printf(" Sensor mutex destroyed\n");
    }
}

/**
 * @description: 获取传感器访问锁
 * @param wait_time 等待时间(毫秒), WAIT_FOREVER表示无限等待
 * @return 成功: 0, 失败: -1
 */
int SensorLock(int32_t wait_time)
{
    if (sht20_mutex == -1) {
        printf(" Sensor mutex not initialized\n");
        return -1;
    }

    int32_t result = UserMutexObtain(sht20_mutex, wait_time);
    if (result != 0) {
        printf(" Failed to obtain sensor lock (Error: %d)\n", result);
        return -1;
    }

    return 0;
}

/**
 * @description: 释放传感器访问锁
 * @return 成功: 0, 失败: -1
 */
int SensorUnlock(void)
{
    if (sht20_mutex == -1) {
        printf(" Sensor mutex not initialized\n");
        return -1;
    }

    int32_t result = UserMutexAbandon(sht20_mutex);
    if (result != 0) {
        printf(" Failed to release sensor lock (Error: %d)\n", result);
        return -1;
    }

    return 0;
}

/**
 * @description: 检查传感器锁是否可用
 * @return 可用: 1, 不可用: 0

```

```

*/
int IsSensorLockAvailable(void)
{
    return (sht20_mutex != -1) ? 1 : 0;
}

```

用法如下所示，先尝试获取锁，如果获取不到锁则不执行物理量获取的过程，直接跳过。

```

if (SensorLock(LOCK_TIMEOUT_MS) == 0){
    struct SensorQuantity* temp = GetTempQuantity();
    temperature = SensorQuantityReadValue(temp);
    SensorQuantityClose(temp);
    SensorUnlock();
    if (temperature > 0) {
        printf("Temperature : %d.%d C\n", temperature/10,
temperature%10);
        sensor_data.temperature = temperature/10.0f;
    }
    else {
        printf("Temperature : %d.%d C\n", -temperature/10, -
temperature%10);
        sensor_data.temperature = -temperature/10.0f;
    }
}
}

```

## 温度读取任务

在传感器读取任务中，我们需要先获取物理量的句柄，然后通过这个句柄去操作传感器，获得数据，原始数据的转换过程已经在驱动层实现，这里是抽象后的应用层实现，充分表明了XIUOS抽象架构的优点——应用层开发和底层实现解耦。

同时，在读数据时，记得加锁，防止缓冲区出问题。

```

/**
 * @description: 温度传感器任务函数
 * @param parameter - 任务参数
 */
void TemperatureTask(void *parameter)
{
    printf(" Temperature sensor task started (ID: %d)\n", UserGetTaskID());
    int32_t temperature;
    while (temperature_task_run) { // 死循环
        printf("\n=== Temperature Measurement ===\n");

        if (SensorLock(LOCK_TIMEOUT_MS) == 0){
            struct SensorQuantity* temp = GetTempQuantity();
            temperature = SensorQuantityReadValue(temp);
            SensorQuantityClose(temp);
            SensorUnlock();
            if (temperature > 0) {
                printf("Temperature : %d.%d C\n", temperature/10,
temperature%10);
                sensor_data.temperature = temperature/10.0f;
            }
        }
    }
}

```

```

    }
    else {
        printf("Temperature : %d.%d C\n", -temperature/10, -
temperature%10);
        sensor_data.temperature = -temperature/10.0f;
    }
}
UserTaskDelay(500);
}
}
}

```

**注意！**

**在这里需要注意的是!!!**，在下面这一处代码中，temperature这个物理量在读取之后需要SensorQuantityClose(temp)，因为在GetTempQuantity中有一个open的操作，在多传感器的情况下，需要先close掉当前打开的传感器物理量才能正常open其他的传感器，否则正在open的传感器将会一直霸占I2C总线，其他传感器不能正常发送和接收命令和数据。

同理可得，如果需要在IO34和35接口同时并联其他传感器或者电机等控制器，需要先close，然后再控制gpio引脚。

```

struct SensorQuantity* temp = GetTempQuantity();
    temperature = SensorQuantityReadValue(temp);
    SensorQuantityClose(temp);
    SensorUnlock();

```

测试结果如下所示

```

=====
Application Completed Successfully
=====

Humidity : 39.4 %RH
=== Temperature Measurement Cycle 2 ===
Temperature : 28.6 C
=== Humidity Measurement Cycle 2 ===
Humidity : 41.5 %RH
=== Temperature Measurement Cycle 3 ===
Temperature : 28.6 C
=== Humidity Measurement Cycle 3 ===
Humidity : 40.9 %RH
=== Temperature Measurement Cycle 4 ===
Temperature : 28.7 C
=== Humidity Measurement Cycle 4 ===
Humidity : 40.4 %RH
=== Temperature Measurement Cycle 5 ===
Temperature : 28.7 C
=== Humidity Measurement Cycle 5 ===
Humidity : 41.5 %RH

```

## 湿度读取任务

湿度读取用的传感器和温度是同一个，此处不再赘述。

```

/**
 * @description: 湿度传感器任务函数
 * @param parameter - 任务参数
 */

```

```

void HumidityTask(void *parameter)
{
    printf(" Humidity sensor task started (ID: %d)\n", UserGetTaskID());

    int32_t humidity;
    while (humidity_task_run) { // 死循环
        printf("\n=== Humidity Measurement ===\n");
        if (SensorLock(LOCK_TIMEOUT_MS) == 0){
            struct SensorQuantity *humi = GetHumiQuantity();
            humidity = SensorQuantityReadValue(humi);
            SensorQuantityClose(humi);
            SensorUnlock();
            printf("Humidity : %d.%d %%RH\n", humidity/10, humidity%10);
            sensor_data.humidity = humidity/10.0f;
        }
        UserTaskDelay(500);
    }
}

```

## 光照强度读取任务

光照强度读取同温湿度读取，可以看到代码结构和上述的温湿度读取代码近乎一样，这依靠XIUOS的抽象层设计。

```

void LightTask(void *parameter)
{
    printf(" Light sensor task started (ID: %d)\n", UserGetTaskID());
    int32_t light_intensity;

    while (light_task_run /* && cycle_count < SENSOR_RUN_CYCLES */) { // 死循环
        printf("\n=== Light Measurement ===\n");

        if (SensorLock(LOCK_TIMEOUT_MS) == 0){
            struct SensorQuantity* light = GetLightQuantity();
            light_intensity = SensorQuantityReadValue(light);
            SensorQuantityClose(light);
            SensorUnlock();

            if (light_intensity >= 0) {
                printf("Light Intensity : %d.%d lx\n", light_intensity/10,
light_intensity%10);
                sensor_data.light_intensity = light_intensity/10.0f;
            }
            else {
                printf("Light sensor read failed: %d\n", light_intensity);
                sensor_data.light_intensity = -1.0f; // 错误值标记
            }
        }
        /* 任务延迟0.5秒 */
        UserTaskDelay(500);
    }
}

```

测试结果如下所示

```

28
29
30 == Light Measurement Cycle 2 ==
31 Light Intensity : 13.3 Lx
32
33 == Light Measurement Cycle 3 ==
34 Light Intensity : 13.3 Lx
35
36 == Light Measurement Cycle 4 ==
37 Light Intensity : 110.8 Lx
38
39 == Light Measurement Cycle 5 ==
40 Light Intensity : 115.8 Lx
41
42 == Light Measurement Cycle 6 ==
43 Light Intensity : 149.1 Lx
44
45 == Light Measurement Cycle 7 ==
46 Light Intensity : 158.3 Lx
47
48 == Light Measurement Cycle 8 ==
49 Light Intensity : 153.3 Lx
50
51 == Light Measurement Cycle 9 ==
52 Light Intensity : 153.3 Lx
53
54 == Light Measurement Cycle 10 ==
55 Light Intensity : 153.3 Lx
56
57 == Light Measurement Cycle 11 ==
58 Light Intensity : 153.3 Lx
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
10
```

## 实时认知——人在识别任务

在人在识别任务中，调用了

APP\_Framework/Framework/knowning/kpu/k210\_yolov2\_detect\_procedure/k210\_yolov2\_detect.c文件下的k210\_detect(char \*json\_file\_path)函数，这个函数能调用摄像头进行人脸识别，我们在其中添加了一个全局变量，用于存储人物数量，然后在人在识别任务的这个文件中extern这个全局变量，即可在此获取人物数量状态，实现感知是否有人的功能。

```

/**
 * @description: k210人脸识别任务函数
 * @param parameter - 任务参数
 */
void DetectTask(void *parameter)
{
    printf(" K210 detect task started (ID: %d)\n", UserGetTaskID());
    k210_detect("face.json");
}

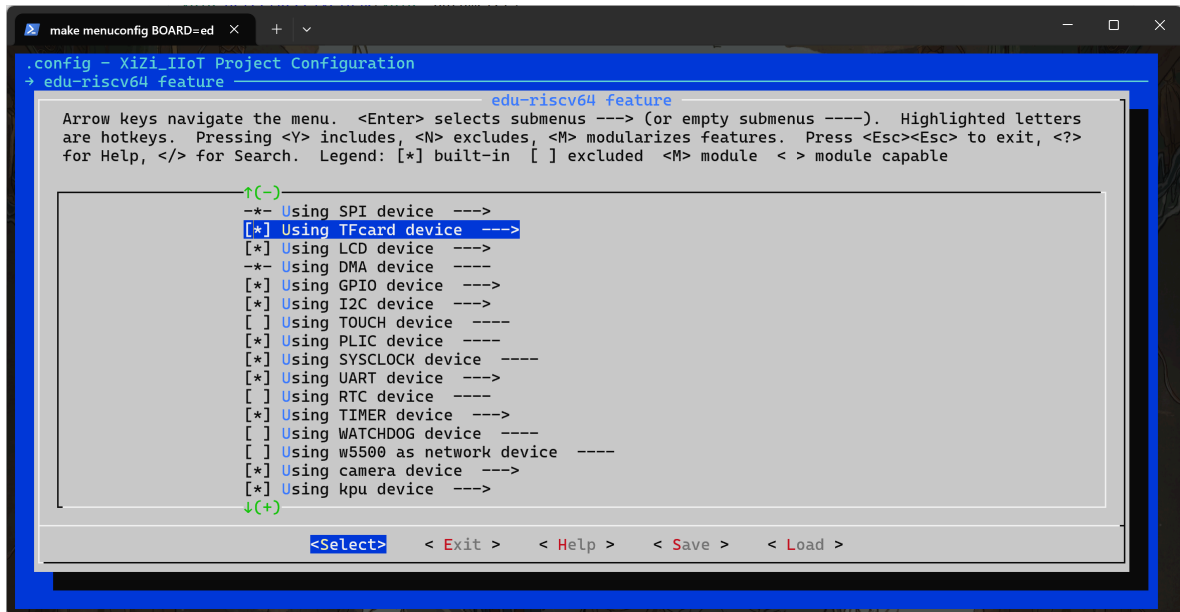
/**
 * @description: 用来接收k210人脸识别任务函数运行时产生的结果 object_exist_or_not
 * @param parameter - 任务参数
 */
void DetectReceiveTask(void *parameter)
{
    printf(" Receive detect task started (ID: %d)\n", UserGetTaskID());
    while (detect_task_run) {
        printf("\n=== Detect Measurement ===\n");
        printf("Data: existing_object_count: %d\n", existing_object_count);
        sensor_data.person_present = existing_object_count > 0 ? 1 : 0;
        UserTaskDelay(500);
    }
}

```

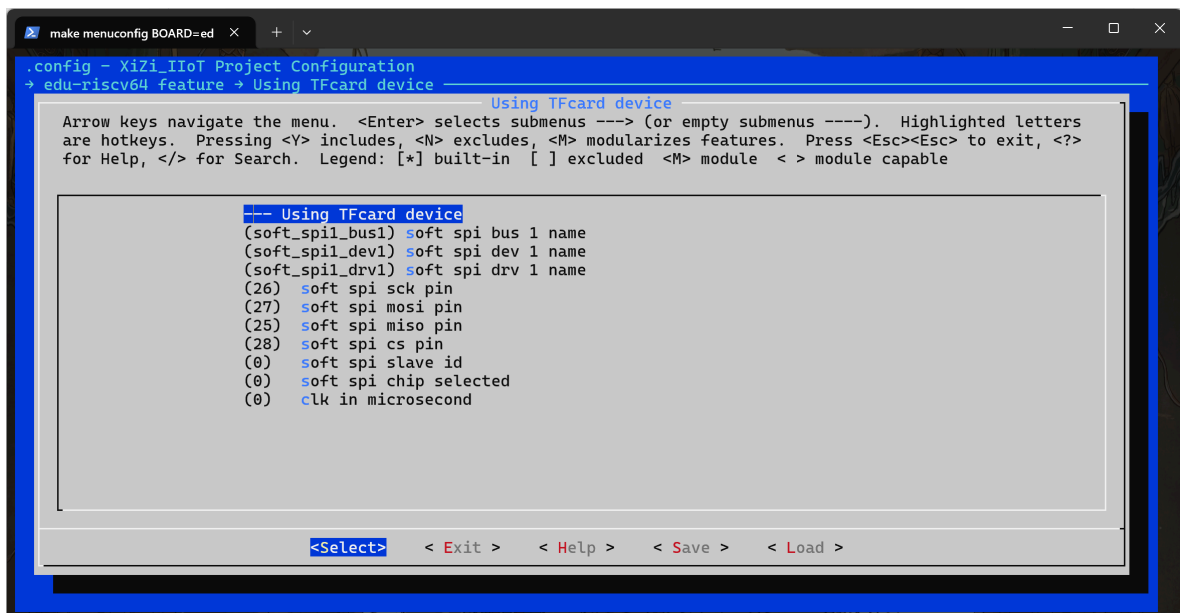
```
}  
}
```

要使用摄像头和LCD屏幕，以及导入TFcard的模型，需要按照以下步骤配置。

勾选以下配置

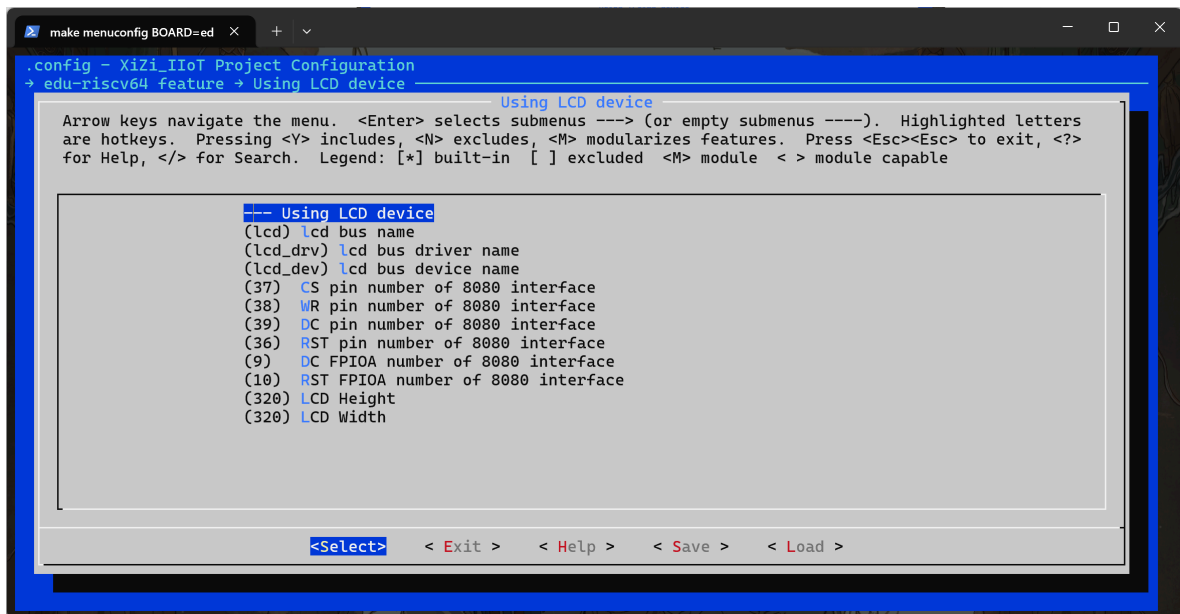


TFcard中的配置

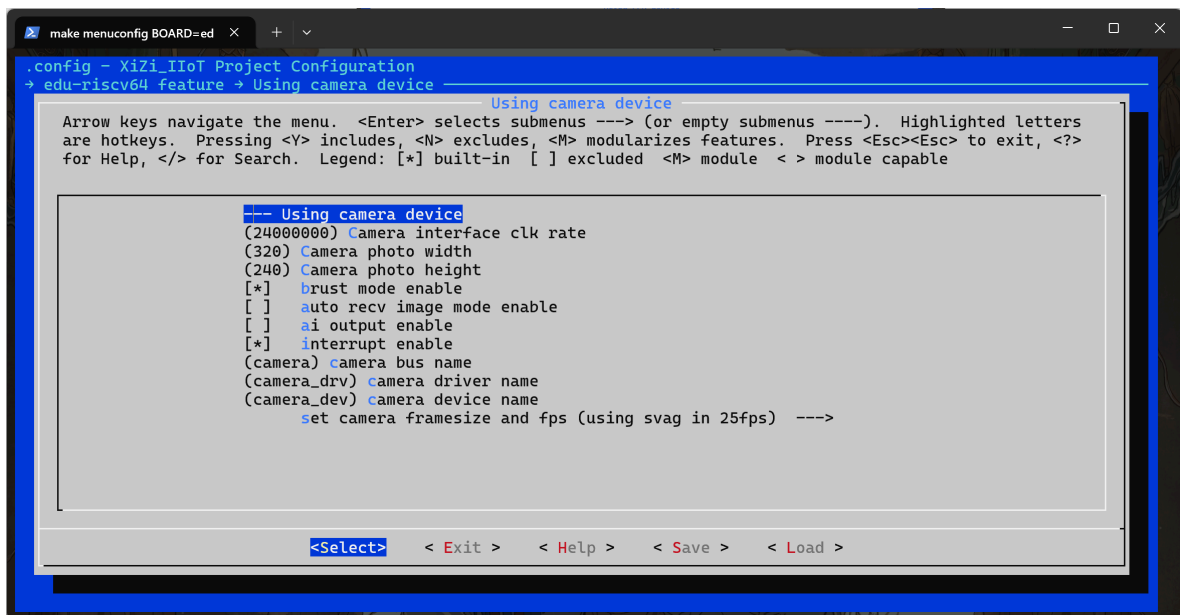


LCD中的配置

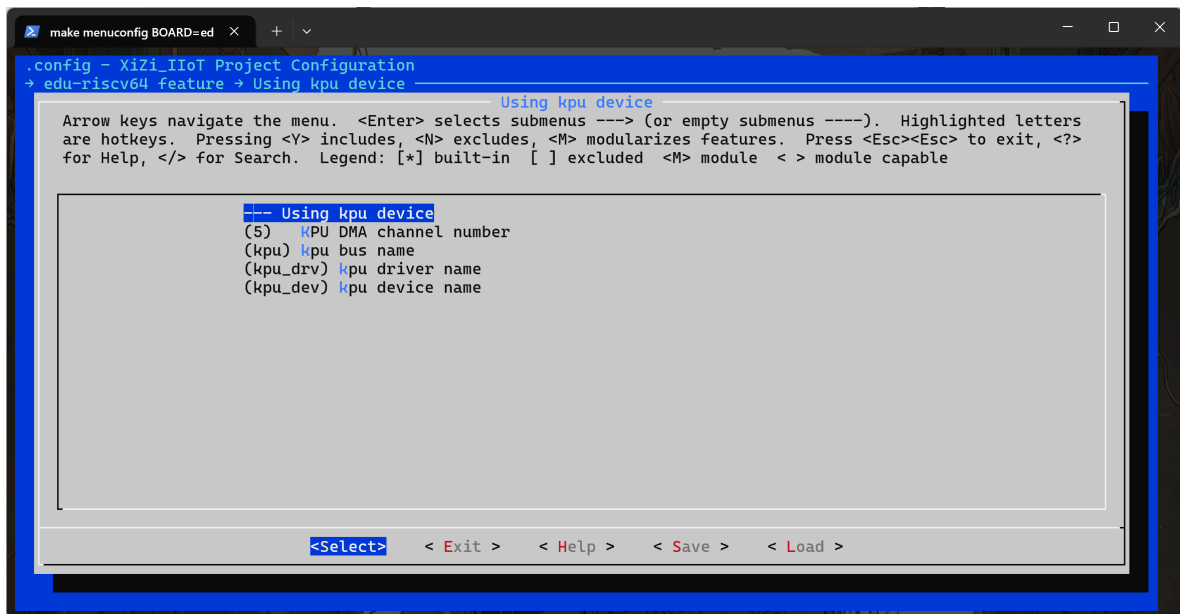




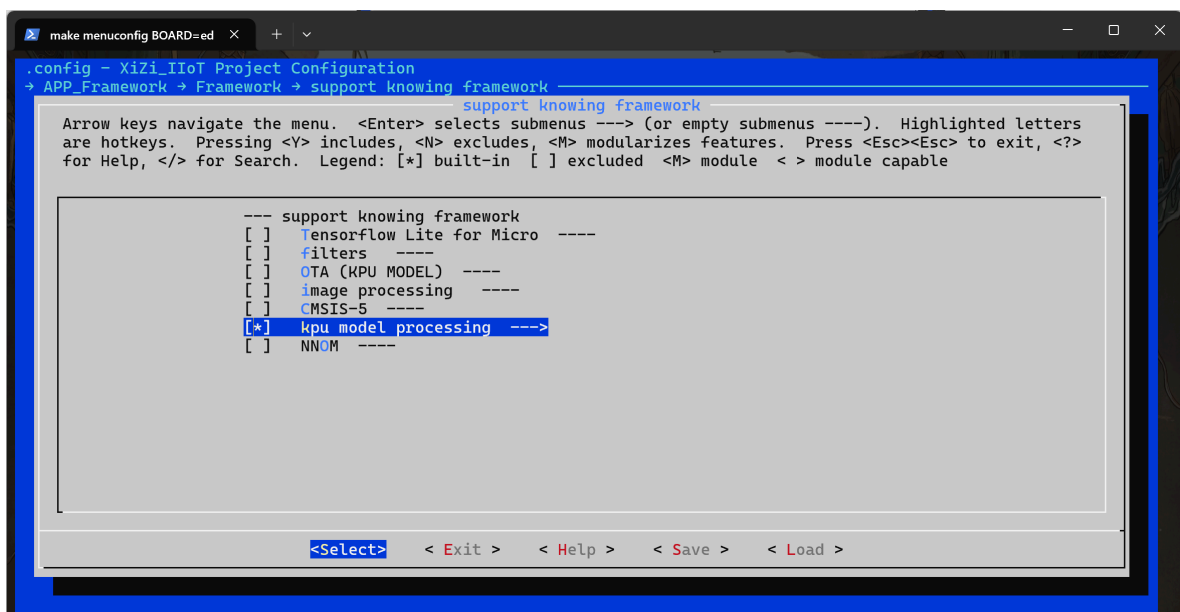
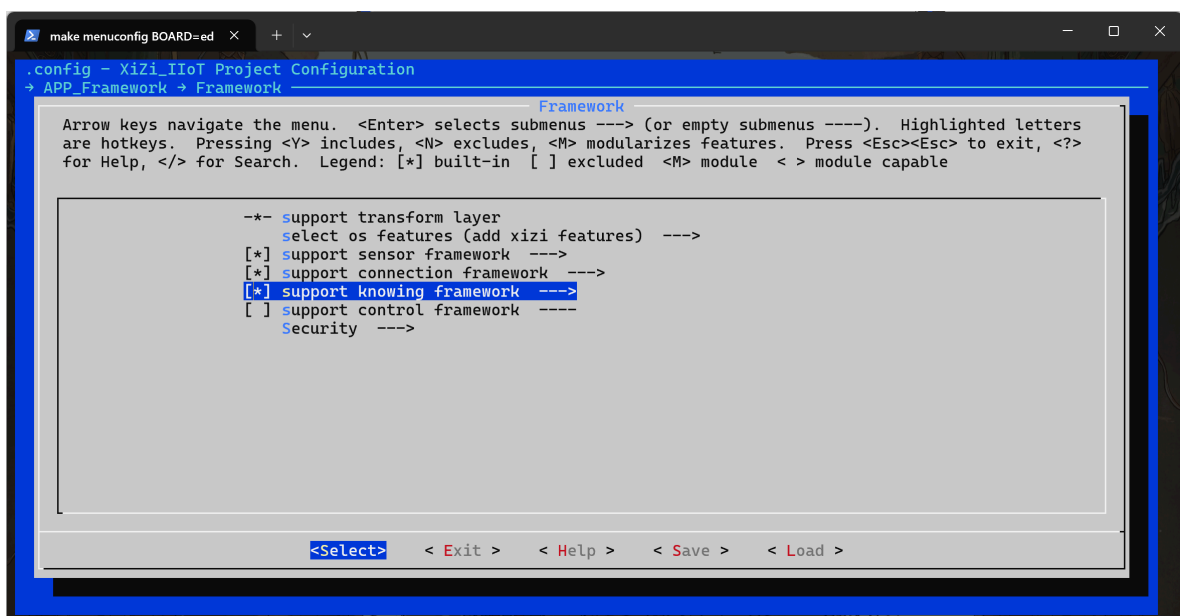
## camera配置

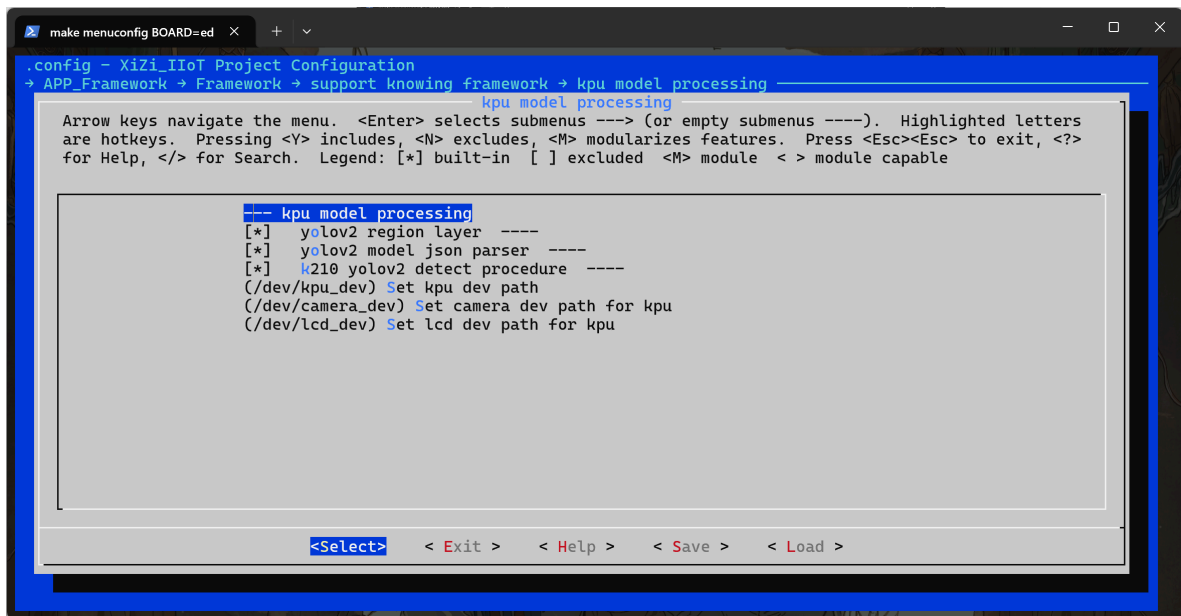


## kpu配置



勾选knowing framework





## MQTT消息发送任务

在本系统开发中，需要使用MQTT作为消息中转的中间件，板端和云端都作为MQTT的客户端，MQTT的服务端使用Mosquitto软件。

板端的数据通过WIFI发送给MQTT服务端，作为生产者。云端订阅相应的主题，消费板端生产的数据，作为消费者。

在各个任务之间，传感器数据是以全局变量的形式进行数据的交互的。在传感器读取数据后，会将最新数据更新到全局变量中，如下所示。

```
sensor_data.humidity = humidity/10.0f;
```

下方是读取所有传感器数据的代码。

```
// 从全局变量获取传感器数据
SensorData sensor_data = GetSensorDataFromQueue();

SensorData GetSensorDataFromQueue(void)
{
    sensor_data.person_present = CheckPersonPresence(); // 检查是否有人
    sensor_data.light_intensity = GetLightIntensity(); // 获取光照强度
    sensor_data.temperature = GetTemperature(); // 获取温度
    sensor_data.humidity = GetHumidity(); // 获取湿度

    return sensor_data;
}
```

下方代码是MQTT客户端任务代码，实现了传感器数据向MQTT服务端的持续发送。在和MQTT服务器建立连接时，首先需要和MQTT服务器所在的ip和端口建立TCP连接，然后根据MQTT协议发送消息进行客户端和服务端的连接，连接成功后即可根据MQTT协议发送消息至MQTT服务器，另一个客户端在订阅了对应主题后，服务器会将消息推送至客户端，这个客户端是我们的云端控制程序。

## 注意!

在这里需要注意，TCP建立连接需要配置这一行代码，配置adapter的socket协议，《手册》没说！！否则会建立不了TCP连接，这里踩坑大约花了5h，请各位引以为戒。

```
g_mqtt_adapter->socket.protocol = SOCKET_PROTOCOL_TCP;
```

```
void MqttEdgeDeviceTask(MqttServerAddr* mqtt_server_addr)
{
    int ret;
    DeviceState current_state = STATE_NO_PERSON;
    DeviceState previous_state = STATE_NO_PERSON;

    // 查找WiFi适配器
    g_mqtt_adapter = AdapterDeviceFindByName(ADAPTER_WIFI_NAME);
    if (!g_mqtt_adapter) {
        lw_print("Failed to find WiFi adapter\n");
        return;
    }
    g_mqtt_adapter->socket.protocol = SOCKET_PROTOCOL_TCP;

    const char *client_id = "xiuos_device_001";
    const char *subscribe_topic = "iot/devices/#";
    const char *status_topic = "iot/devices/status";
    const char *ping_topic = "iot/devices/ping";

MQTT_CONNECT:
    // 1) TCP连接到 MQTT 服务器
    lw_print("Connecting MQTT TCP: %s:%s ...\n", mqtt_server_addr->ipv4,
mqtt_server_addr->port);
    ret = AdapterDeviceConnect(g_mqtt_adapter, CLIENT, mqtt_server_addr->ipv4,
mqtt_server_addr->port, IPV4);
    if (ret < 0) {
        lw_print("TCP connect failed: %d\n", ret);
        PrivTaskDelay(3000);
        goto MQTT_CONNECT;
    }
    lw_print("TCP connected to %s:%s\n", mqtt_server_addr->ipv4,
mqtt_server_addr->port);

    // 2) 发送 MQTT CONNECT (参考 MqttTest)
    {
        MQTTPacket_connectData data = MQTTPacket_connectData_initializer;
        uint8_t conn_buf[256];
        int conn_len;

        data.clientID.cstring = (char *)client_id;
        data.keepAliveInterval = 60;
        data.username.cstring = NULL;
        data.password.cstring = NULL;
        data.MQTTVersion = 4;      // MQTT 3.1.1
        data.cleansession = 1;

        conn_len = MQTTSerialize_connect(conn_buf, sizeof(conn_buf), &data);
        if (conn_len <= 0) {
```

```

        lw_print("MQTT CONNECT serialize failed\n");
        AdapterDeviceClose(g_mqtt_adapter);
        PrivTaskDelay(3000);
        goto MQTT_CONNECT;
    }
    AdapterDeviceSend(g_mqtt_adapter, conn_buf, conn_len);
    lw_print("MQTT CONNECT sent (%d bytes)\n", conn_len);
    PrivTaskDelay(500);
}

// 3) 订阅主题 (QOS0)
{
    uint8_t sub_buf[200];
    MQTTString topic = {.cstring = (char *)subscribe_topic};
    int qos0 = 0;
    int sub_len = MQTTSerialize_subscribe(sub_buf, sizeof(sub_buf), 0, 1 /*
packet id */, 1, &topic, &qos0);
    if (sub_len > 0) {
        AdapterDeviceSend(g_mqtt_adapter, sub_buf, sub_len);
        lw_print("Subscribe %s success (len=%d)\n", subscribe_topic,
sub_len);
    } else {
        lw_print("Subscribe %s failed to serialize\n", subscribe_topic);
    }
}

// 主循环
uint8_t no_mqtt_msg_exchange = 1;

// 接收缓冲区
uint8_t recv_buf[512];

while (1) {
    UserTaskDelay(500);

    // 尝试接收消息并反序列化 PUBLISH 载荷
    ssize_t recv_len = AdapterDeviceRecv(g_mqtt_adapter, recv_buf,
sizeof(recv_buf));
    if (recv_len > 0) {
        unsigned char dup, retained;
        int qos;
        unsigned short packetid;
        MQTTString topicName;
        unsigned char *payload = NULL;
        int payloadlen = 0;

        int ok = MQTTDdeserialize_publish(&dup, &qos, &retained, &packetid,
&topicName, &payload, &payloadlen,
recv_buf, (int)recv_len);
        if (ok == 1 && payload && payloadlen > 0) {
            lw_print("Received MQTT publish: topic=.%s, payload_len=%d\n",
topicName.lenstring.len, topicName.lenstring.data,
payloadlen);
            ProcessMqttMessage(payload, payloadlen);
            no_mqtt_msg_exchange = 0;
        } else {

```

```

        // 非 PUBLISH 或解析失败，忽略
    }
}

// 从全局变量获取传感器数据
SensorData sensor_data = GetSensorDataFromQueue();

// 发布设备状态（不再使用 AdapterDeviceMqttSend，改用 QOS0 发布）
{
    cJSON *root = cJSON_CreateObject();
    cJSON_AddStringToObject(root, "device_id", client_id);

    cJSON *sensors = cJSON_CreateObject();
    cJSON_AddBoolToObject(sensors, "person_present",
sensor_data.person_present);
    cJSON_AddNumberToObject(sensors, "light_intensity",
sensor_data.light_intensity);
    cJSON_AddNumberToObject(sensors, "temperature",
sensor_data.temperature);
    cJSON_AddNumberToObject(sensors, "humidity", sensor_data.humidity);
    cJSON_AddItemToObject(root, "sensor_data", sensors);

    const char *state_str[] = {
        "no_person", "person_enter", "person_dark", "person_bright",
        "high_humidity", "high_temperature", "normal"
    };
    cJSON_AddStringToObject(root, "device_state",
state_str[current_state]);

    char *json_str = cJSON_PrintUnformatted(root);
    lw_print("Publishing: %s\n", json_str);

    if (AdapterMQTTPublish_QOS0(g_mqtt_adapter, (char *)status_topic,
(uint8_t*)json_str) == 0) {
        lw_print("Publish success\n");
    } else {
        lw_print("Publish failed\n");
    }

    cJSON_free(json_str);
    cJSON_Delete(root);
}

}

}

```

